DSH / COMPONENTIZATION

DSH 组件化接入指南

复用公共能力，适配目标宿主，验证实际调用

SDK · CLI · MCP · HTTP · Skills & Agents · ACP

公共接口选择 / 插件与 Bundle / DSH 安装启动
市场来源与收录 / 项目接入建议 / 模板与接入示例

五项轻量要求 / 按需扩展 / AI 实施与验收

讨论稿 v0.9 | 2026-10-15

版本：讨论稿 v0.9 · 2026-10-15

操作指引

把项目仓库、接入卡 [S01]和轻量验收表 [S02]交给 AI。默认目标是让同一份业务能力适配不同 Agent CLI，减少重复开发和维护。先复用已有 SDK、CLI、MCP 或资源；只有实际需要时才新增 HTTP 服务。

步骤	怎么做	交付结果
1. 选范围与入口	第 2-3 章：选首个场景，保留能用的公共入口	简短接入卡
2. 复用与适配	第 4-6 章：维护一份业务逻辑，只处理必要的宿主差异	共用实现与薄适配/配置
3. 安装使用	DSH 已发布包看 7.1；HTTP 示例看 7.2-7.8；CLI/SDK 看 5.5，打包看 7.9	版本明确、可安装/加载的交付物
4. 验证与交付	第 11 章、附录 B：五项轻量要求，按实际行为补扩展	调用结果、简短 README 与验收表
可选：发布收录	确有需要时按 6.15-6.17 发布，再提交市场	发布与收录状态

完成标准：L01 共用实现、L02 调用约定、L03 薄适配、L04 安装使用、L05 实际验证全部通过；实际触发的扩展通过，且结果有证据。 普通本地调用不要求服务化、任务队列、流式 API、独立复核或全套生产运维。

只交付公共能力时，验收其入口；声明完成 DSH 接入时，验证实际 DSH 调用。长手册供 AI 和开发者按需查询，人验收时先看表里的结论、未通过项和证据。八个项目建议见第 8-10 章，完整示例见附录 C。

使用配套文件时保留本文件、`examples/` 和两个模板的目录结构。PDF 用于阅读，Markdown 用于填写；第 7 章命令读取配套文件中的完整示例。HTTP 示例只代表一条路线。

目录

点击条目可跳转到对应页；侧栏书签可展开各章。

1 先分清组件 Plugin Bundle 和 Profile . . . 5
1.1 安装命令、包与配置文件怎样对应 . . . 6
2 为项目选择公共交付形式 . . . 6
2.1 一层薄 Driver 的含义 . . . 7
2.2 按这六个问题做第一次选择 . . . 7
3 每个项目先填一张接入卡 . . . 8
3.1 一张已填写的接入卡 . . . 8
4 五项轻量要求与接入流程 . . . 9
4.1 公共接口最少约定哪些内容 . . . 9
4.2 目录怎么拆 . . . 9
5 按现有入口接入 DSH . . . 10
5.1 HTTP API 组件 . . . 10
5.2 MCP 工具组件 . . . 10
5.3 Skills 与 Agent 资源 . . . 11
5.4 ACP 完整 Agent . . . 13
5.5 已有 CLI 或 SDK，直接复用 . . . 13
6 Plugin 与 Bundle 的拆分、装配和发布 . . . 14
6.1 Plugin 与 Bundle 不是二选一 . . . 14
6.2 什么时候拆成多个 Plugin . . . 15
6.3 不按哪些方式拆分 . . . 16
6.4 多个 Plugin 放一个包还是多个包 . . . 16
6.5 单插件与 Bundle 同包 . . . 16
6.6 同包导出多个 Plugin . . . 17
6.7 多个 Plugin 包组成一个 Bundle . . . 18
6.8 一个 Bundle 组合到什么程度 . . . 19
6.9 发布前的决策与交付检查 . . . 20
6.10 复制一个确实有两个插件的 Bundle . . . 20
6.11 三类依赖不要混在一起 . . . 21
6.12 市场发现、包分发与 DSH 加载是三件事 . . . 21
6.13 非官方市场、目录与发现工具一览 . . . 22
6.14 DeepSeek.club 应怎样理解和提交 . . . 24
6.15 从本地交付物发布到 npm 或 GitHub . . . 24
6.16 从已发布的包到市场收录 . . . 26
6.17 发布与收录的记录方式 . . . 26
7 从打包到 DSH 启动 . . . 27
7.1 DSH 原生安装：直接从发布源获取 . . . 27
7.2 准备同一个运行时与独立接入目录 . . . 28
7.3 启动独立服务并打包 . . . 30
7.4 创建 Profile 并安装 . . . 30
7.5 配置地址并检查合成结果 . . . 31

7.6 启动、选择模型并完成首次调用 . . . 31
7.7 验证失败、可选开关与退出 . . . 32
7.8 换成业务组件并交付他人 . . . 33
7.9 其他路线的打包和验收 . . . 33
8 八个项目的接入与拆分总览 . . . 35
8.1 当前形态、首次接入与验收 . . . 35
8.2 各项目的初始插件边界 . . . 36
9 Insight 与 AgentLineage 的联合方案 . . . 37
9.1 完整材料与增量材料都需要 . . . 37
9.2 建议共同使用的处理链 . . . 37
9.3 什么时候增加 Stream API . . . 37
9.4 现有能力与本次待做的连接 . . . 38
9.5 联合方案按什么顺序实现 . . . 38
10 各项目的具体注意事项 . . . 39
10.1 CPX . . . 39
10.2 SIFT . . . 40
10.3 ECAT . . . 40
10.4 DT Verify . . . 40
10.5 Layr . . . 41
10.6 EngramForge . . . 41
11 团队交付与验收 . . . 42
11.1 怎样判定组件化完成 . . . 42
11.2 交给 AI 什么，最终收回什么 . . . 42
11.3 失败时从哪里查 . . . 43
12 本版依据和待确认项 . . . 43
附录 A 组件接入卡 . . . 45
A.1 基本信息 . . . 45
A.2 最小样例 . . . 45
A.3 宿主接入与交付 . . . 45
A.4 按需补充 . . . 45
A.5 交付记录 . . . 46
附录 B 验收记录 . . . 47
B.1 本次范围 . . . 47
B.2 五项轻量要求 L01-L05 . . . 47
B.3 触发时才增加的检查 . . . 47
B.4 完成判定与证据 . . . 48
B.5 给 AI 的实施与验收指令 . . . 48
附录 C 最小接入示例 . . . 49
C.1 HTTP 单插件 Bundle . . . 49
C.2 MCP 配置型 Bundle . . . 51
C.3 Skills 资源 Bundle . . . 52
C.4 HTTP 与 Skill 同包多插件 . . . 54
C.5 HTTP 示例服务 . . . 56
附录 D 资料与源码定位 . . . 57

1 先分清组件 Plugin Bundle 和 Profile

名称	负责什么	例子
组件	一项可被独立复用的业务能力及其输入输出	会话分析、证据查询、执行审计
公共入口	多个调用方共用的能力入口，不等于公开网络服务	SDK、CLI、MCP、HTTP、Skills & Agents、ACP
Driver	处理某个宿主的接入差异	参数映射、工具注册、取消和结果呈现
DSH Plugin	在 DSH 中加载并执行的扩展代码，或已有的协议接入插件	HTTP 工具插件、MCP client、skill provider
DSH Bundle	在包清单声明 `dsh.bundle.patch` 的装配单元，可经 npm、Git 或 tarball 分发	装配 Insight 和 AgentLineage 的多个插件
DSH Profile	本次 DSH 运行选择的 bundle 列表与覆盖配置	包含 base、Web、业务 bundle 的团队 profile

启动与调用是两条链：

```text
启动：dsh --profile team-web
      → 读取 profile 的 bundle 列表
      → 合成 bundle patch 和覆盖配置
      → 加载 plugin，登记工具、资源或执行后端

调用：DSH Agent
      → 使用工具、加载 skill 或派发 agent
      → 公共能力入口
      → 组件核心处理
      → 返回结构化结果、产物和证据
```

一个 bundle 可以加载一个或多个插件；一个插件可以注册多个工具。同一个 npm 包可以同时提供 plugin 代码和 bundle patch。**Plugin 与 Bundle 不是二选一：前者划分运行职责，后者提供安装与装配配置。** 项目、包、插件、Bundle 和市场条目的数量不需要相等。

市场可以把“CPX 插件”展示为一张卡片；用户实际安装的是一个带 Bundle 配置的包，内部可能只有一个 capture Plugin。社区使用 plugin 作为扩展的统称，不表示这些交付物没有 Bundle。市场来源与发布流程见第 6.12-6.17 节。

Bundle 是 DSH 的安装与装配单位。它不会因为写了几个服务地址，就自动启动 Python 服务、数据库、代理或设备。谁负责启动这些进程，必须另行说明。

1.1 安装命令、包与配置文件怎样对应

`dsh plugin --profile web add '@team/component-demo@0.1.0'` 的直接安装对象是一个 npm 包。包声明了 Bundle 后，DSH 才把它加入 `web` 的装配列表。包名是示例，实际操作使用作者发布的名称和版本。

位置	放什么	安装与使用时怎样处理
组件包的 `package.json`	npm 名称/版本、Plugin 导出、`dsh.bundle.patch`	作者编写；安装器读取其 Bundle 声明
组件包的 `cordis.patch.yml`	默认插入的 Plugin 实例与配置	随包安装；安装命令不改写它
Profile 的 `package.json`	依赖和 `dsh.profile.bundles`	安装命令更新；决定这个 Profile 选择哪些 Bundle
Profile 的 `cordis.patch.yml`	实际服务地址、功能开关等部署配置	用户或管理界面修改；覆盖组件默认值

包内 patch 和 Profile patch 可以同名，属于两个不同目录。更换自己的地址或开关时改 Profile 配置；改组件默认装配时改源码包、升版本并重新发布。安装器还会维护依赖目录和锁文件，不能只复制一份 YAML 就认为包已经安装。

包名、插件模块名、实例 ID、工具名也分别有用途。例如 `@team/component-demo` 是包及其默认模块入口，`team.component.demo` 是 patch 用来定位实例的 ID，`component_health` 是 Agent 调用的工具名。改名时核对清单、patch、代码和 README；它们不要求同名。

2 为项目选择公共交付形式

先保留能用的入口，再处理宿主差异。公共接口可以是本地函数/SDK、CLI 命令、stdio MCP 或资源包；不要求公网服务，不要求常驻进程，也不要求同时提供所有形式。只有远程调用、独立运行或共享服务等真实需求出现时，再评估 HTTP。

项目主要提供什么	优先保留或建设什么	DSH 如何接入	主要成本
查询、分析、本地数据处理	先复用已有 SDK/CLI；需要独立服务或远程调用时再用 HTTP	SDK/CLI 薄适配；已有 HTTP 则接工具	入口依赖与输入输出；选服务后才增加部署成本
已经能被 Agent 调用的独立工具	保留现有 MCP；本地可用 stdio，不必新建 HTTP 服务	配置型 bundle；按需补宿主差异	启动/连接配置、输出与宿主兼容
本地确定性脚本或成熟 CLI	CLI/库，稳定 JSON、退出码；需要时包成 MCP	skill 指引调用，或受管工具桥接	环境安装、路径、进程生命周期
工作方法、提示词、模板和脚本	Skills 资源包	skill-filesystem 或自定义 provider	资源路径、工具词表、脚本环境
角色定义，复用宿主模型和工具	Agent 资源及平台映射规则	preset/persona，或确有需要时自定义角色路由	角色、工具权限、父子任务映射
已有独立 Agent Loop、模型、会话和工具	完整 Agent，适合时提供 ACP	DSH ACP 子 Agent 后端	会话、授权、取消、成本及产物回传
代理、采集器、设备 runner 等常驻能力	独立进程或服务，加控制接口	启动配置或生命周期插件，另配必要工具	启动时序、资源占用、退出与恢复

例如，同一个 SIFT 交付物可以包括 CLI、一个 MCP 包装层和操作说明 skill；不需要为 skill 正文再建一个 HTTP 服务。同一个 ECAT 项目可以把评估内核做成可调用能力，把角色和流程保留为资源，并单独决定完整迁移 Agent 的执行方式。

2.1 一层薄 Driver 的含义

Driver 只处理所选宿主需要的工具/资源注册、配置、参数转换、调用和结果展示。涉及耗时调用、凭据或自有资源时，再处理对应的超时、取消与清理。业务分析、评分、会话解析、经验管理仍由组件维护；只需协议配置时不必新增 Driver 代码。

HTTP 客户端、MCP 包装层和 DSH 注册代码可以分文件或分包；“一层”约束的是职责，不能为了减少文件数，把业务逻辑搬进插件。宿主已经支持所选 MCP 或 ACP 方式时，优先复用已有接入插件。不要默认串成 `DSH → 自定义 Driver → MCP → HTTP → 第二套业务实现`。

2.2 按这六个问题做第一次选择

先选一个本次交付场景，再找现有入口。首版只做完成该场景所需的最小组合。

顺序	先问什么	如果是，首版怎么做
1	主要交付方法、角色、提示词、模板或脚本?	走 5.3 的资源路线；复用正文和引用，不为传 Markdown 新建 API
2	已有可用 SDK、函数或 CLI?	优先复用，按 5.5 直接调用或薄包装；只补输入输出和错误说明
3	已有可用 MCP?	走 5.2，用宿主已有 client；本地 stdio 可由宿主启动进程
4	已有 HTTP 服务，或确实需要远程/共享服务?	走 5.1，复用 HTTP 入口；服务化作为本次选择，不作为其他项目门槛
5	已有完整 Agent Loop、模型、会话和工具?	保留运行时，按 5.4 评估 ACP；普通角色资源不需要这一层
6	还涉及常驻、长任务、写入或增量数据?	在已选路线外补对应 E 项；按行为设计，不自动新增 HTTP、队列或流式协议

选择结果写成一句话，例如：“SIFT 首版复用现有 CLI，用 Skill 说明调用方法；需要固定工具参数时再加桥接。”Insight 已有服务、AgentLineage 当前 DSH 插件消费 HTTP，都可以保留现状；这不要求其他项目照做。

填清入口、目标宿主、首个场景、必要依赖与暂缓项后进入第 4 章。第 5 章只读所选路线；如果还没有可复现的成功调用，先完成组件自身调用或资源使用。

3 每个项目先填一张接入卡

不需要先提交完整设计。以下八项足够开始分类：

1. 仓库、负责人和当前版本。
2. 一句话用途：给什么输入，完成什么事，返回什么。
3. 一个最小调用实例与实际结果。
4. 现有入口：库、CLI、HTTP、MCP、skills、agents、ACP 中有哪些。
5. 状态与时间：一次调用结束，还是持续采集、长任务、多轮交互。
6. 执行依赖：模型、运行时、数据库、文件系统、编译器、设备。
7. 副作用与权限：读写哪里，能否修改代码、启动程序、发布内容。
8. 首次接入范围与验收方法。

复制接入卡模板 [S01]的基本信息和最小样例到项目 docs；其他字段只填实际涉及的部分。同步准备轻量验收表 [S02]，实施时记录版本、成功/失败结果。未知项写待补；没有扩展时说明一次即可。团队可按场景安排独立复核，不要求每个首版都设多个审批角色。

3.1 一张已填写的接入卡

以下只按第 7 章的 HTTP 路线示范填写，不是所有组件的字段要求。CLI/SDK/资源包应换成自己的入口与场景，删除不涉及的服务配置。

字段	本次填写结果
用途	查询独立服务的健康状态，返回结构化 status
现有入口	HTTP GET /health；不需要专门的 Agent Loop
状态与依赖	短请求；服务是独立 Node 进程，DSH 不替它启动
公开能力	component_health 工具；参数为空，服务地址由 profile 配置
首次交付	@team/component-demo npm 包，同时包含一个 plugin 和 bundle patch
为什么先不拆	只有一个可独立使用的查询职责，没有第二套生命周期
后续组合	如需随包使用方法，可增加独立 Skill 加载 plugin；接入指引见第 6.10 节
配置与权限	baseUrl；可选 tokenEnv 仅写变量名；本地示例服务默认不需要 token
取消与失败	请求五秒超时；服务错误、非法结果、取消都不能返回成功
完成证据	服务收到请求，DSH 有真实工具调用和 status 结果；另测停服失败

业务项目用同一张卡替换输入输出、状态和验收场景。负责人可以先确认这张卡，再开始写接入代码。

4 五项轻量要求与接入流程

步骤	做什么	留下什么	对应验收
1 共用实现	确认业务或资源只维护一份，宿主调用它	现有代码/资源位置，必要的边界调整	L01
2 调用约定	选择一个入口，说明输入、输出、常见错误	类型/schema/CLI 帮助及一个例子	L02
3 薄适配	注册、参数转换、调用、结果展示；可直接配置则不写 Driver	小插件、桥接或宿主配置	L03
4 安装使用	说明实际版本、依赖和安装/加载方式	包或源码交付位置、简短 README	L04
5 实际验证	每个声明支持的宿主跑成功与常见失败；仅公共能力则验自身入口	命令/操作、真实结果、验收表	L05

默认不要求新建服务、拆仓库、公开发布、独立复核、包摘要或跨版本回滚专项。实际产生服务、长任务、写入和增量数据时，再按附录 B 选择扩展。交付 DSH 插件时仍需符合 DSH 的包、patch 和加载机制；简化流程不改变宿主契约。

4.1 公共接口最少约定哪些内容

一页说明或现有代码文档即可，不要求另写完整 OpenAPI。

内容	最少写什么
输入	调用者必须给什么；直接引用已有函数参数、CLI 参数或工具 schema
输出	成功返回什么，产物在哪里；已有稳定结构就继续用
常见错误	一个常见失败如何识别，例如异常、非零退出码、工具错误或缺资源
调用例子	一条可运行命令或 SDK 调用，附实际结果
使用条件	版本、必要依赖与配置；涉及写入时注明副作用

按场景补充：真正有服务才写地址与访问边界；慢调用说明超时/中断；有副作用才讨论重复执行与恢复；持续输入才约定位置与断线语义。没有这些行为就不用设计对应功能。

长任务确需后台执行时，可采用 `start → job_id → status/result → cancel`；简单同步调用不要求任务系统。远程 HTTP 上传时传材料内容或已注册材料 ID，本地 CLI/SDK 则可以按自身约定读本地路径，不需要为了统一形式改成上传服务。

4.2 目录怎么拆

以下是团队建议布局，可以按现有项目调整：

```text
component/
  core/                    业务能力、数据与计算
  interfaces/              SDK、CLI、MCP、HTTP、ACP，只保留所需入口
  assets/
    skills/                SKILL.md 与 references/scripts/templates
    agents/                角色正文、元数据和工具映射说明
  integrations/dsh/        薄插件或配置型 bundle
  examples/                输入、预期输出和启动样例
  docs/                    安装、配置、故障、版本、验收
```

不用先改仓库才能采用这套分工。先把依赖方向和入口分清，再决定是否拆包、拆进程或拆仓库。

5 按现有入口接入 DSH

5.1 HTTP API 组件

仅在已有或确需 HTTP 服务时走本节。先验证该服务，再把选定接口注册成 DSH 工具。其他路线可直接跳到 5.2-5.5，不为使用本示例额外做 HTTP。

AgentLineage 已有可参考的 plugin.ts [S03]、client.ts [S04] 和 bundle patch [S05]。其主要模式是：

```ts
// 模式节选；parameters、output 和 client 需要按实际接口定义。
export const inject = ["tools"];
export function apply(ctx, config) {
  const client = createComponentClient(config);
  const tool = defineTool({
    name: "component_query",
    parameters,
    output,
    execute: (args, exec) => client.query(args, exec.signal),
  });
  ctx.effect(() => ctx.tools.register(tool));
}
```

每个工具写清输入、输出和实际副作用。调用方取消要传到请求；服务端异步任务的取消需要独立操作。工具卸载要释放注册和自己拥有的资源。URL、token 变量名、超时属于配置；业务算法不进入插件。

不必把所有 HTTP 接口一对一注册为模型工具。材料上传、自动采集和快照创建可由业务宿主负责，模型只接收查询所需的 ID。AgentLineage 当前就是这种分工：插件查询证据，但不会自动捕获 DSH 会话。

执行顺序：先用组件自身客户端完成一次真实 API 调用，再复制附录 C.1 HTTP 示例的包结构，替换工具 schema 与请求逻辑，最后按第 7.2-7.8 节打包和验收。保留同一份输入及 API 结果，供第 11 章核对 DSH 返回值。示例 health 工具只验证接入方式，业务交付要换成实际能力。

5.2 MCP 工具组件

本地 MCP 可使用 stdio，由宿主启动程序；不要求独立部署网络服务。下面保留已有 Streamable HTTP 服务的配置示例，本地路线按随后 stdio 配置操作。

如果组件已经提供 MCP，优先用 `@deepseek-ai/dsh-mcp-client`。一个仅含 `package.json` 和 `cordis.patch.yml` 的配置型 bundle 就能描述接入，不必再写一套工具注册代码。

```yaml
- insert:
    - id: team.demo.mcp
      name: '@deepseek-ai/dsh-mcp-client'
      config:
        serverName: component_demo
        transport: streamable-http
        url: http://127.0.0.1:8120/mcp
        failOnStartupError: true
```

这里的 URL 是示例。服务已运行、协议和鉴权匹配时，工具名称形如 `mcp__component_demo__ping`。本地 stdio 则配置 `command`、`args`、`env` 和必要的 `cwd`。stdio 程序的 stdout 必须只输出协议消息，日志写 stderr。

`failOnStartupError: true` 表示初始失败时拒绝该插件激活；DSH 对可选插件的启动策略仍可能允许应用继续运行。所以必须实际查工具和调用结果，不能用“Web 打开了”代替 MCP 验收。具体字段见 DSH MCP client [S06]。

MCP 的 Streamable HTTP 是工具协议的传输方式；它不自动提供业务会话的增量采集、去重和补偿。这些需要组件自己的约定。MCP 传输规范 [S07]

可复制的文件见附录 C.2 MCP 示例。

已有本地 stdio Server 时怎样改。 将 MCP 示例中该实例的完整 config 替换为下面结构，不能只把 transport 改为 stdio 后保留 HTTP 字段。命令与文件是占位项，必须先按该 Server 的说明安装并验证。

```yaml
- id: team.demo.mcp
  config:
    serverName: component_demo
    transport: stdio
    command: 'D:/tools/component/.venv/Scripts/python.exe'
    args: ['-m', 'your_component.mcp_server']
    cwd: 'D:/tools/component'
    env: {}
    toolCallTimeoutMs: 60000
    failOnStartupError: true
```

配置好后按第 7.9 节打包、安装和验证，不执行 HTTP 示例的 `/health` 检查。DSH 负责拉起和关闭这个 stdio 子进程；Streamable HTTP 服务则需要单独启动。用 MCP Inspector 或该 Server 自带客户端先确认初始化、tools/list、tools/call，再装 bundle。`GET /health` 返回 JSON 的普通服务不是 MCP Server，不能把第 7 章的示例服务地址填到 `/mcp` 就认为可用。

5.3 Skills 与 Agent 资源

Skills。 保留 `SKILL.md` 及其引用的脚本、模板、资料。DSH 的常规路径是 `dsh-skill` 注册表、`dsh-skill-filesystem` 发现与读取、`dsh-tool-skill` 提供模型调用入口。宿主已有这些服务时直接复用。

作为 bundle 交付时，把资源放进包的 `files`，明确从已安装包位置解析资源目录，再配置 filesystem provider，或注册自定义 provider。不要让资源路径取决于开发机路径或启动时的 cwd。多个 provider 需要唯一名称；要测试同名 skill 的选择结果。

验收不能只看目录：还要加载正文；含相对引用时打开引用，含必需脚本时验证脚本。`skills:` 字段或正文里写了某个工具名，不代表宿主已经安装或授权它。DSH skill-filesystem [S08]

Agent 资源。 先分清“角色与指引”和“独立运行的 Agent”。角色 Markdown 可以映射为 DSH preset/persona，或通过定制插件选择角色再交给 DSH 子 Agent 执行。不要假设任意 `agents/*.md` 都会被 DSH 自动识别。

HMigBot 是已有的定制参考：读取角色文件，注册一个 `hmigbot-migration` 执行后端，工具选择角色后才创建子 Agent。注册 provider 本身不会执行任务。当前 DSH 的 preset、persona、subagent API 仍要按目标版本核对，不能直接复制旧适配代码。

资源包至少写清角色/Skill 的用途、输入输出、加载方式和调用例子。涉及工具、关联 Skills、脚本或委派时，再列出对应依赖、运行环境和权限要求。若要求沿用 DSH 的模型、会话和权限，优先采用宿主内的执行方式。

本地接入：确认 Skill 能被发现。 把附录 C.3 的完整 `component-check/` 目录复制到选定工作区的 `.dsh/skills/` 下，同时保留 `SKILL.md` 和 `references/checklist.md`。只复制正文会使相对引用失效。创建标准 preset 的新会话，请它加载 `component-check` 并读取相对引用。工作区所在的 `.git` 祖先会影响发现根目录；用第 7 章创建的空工作区最容易定位。

随包交付：把 Skill 装入组件包。 附录 C.3 给出完整资源 bundle。它的薄加载器从 `import.meta.url` 解析安装后的 `assets/skills`，复用 filesystem provider，设置唯一 providerName，关闭重复的默认目录扫描。包内带 `SKILL.md` 和 `references/checklist.md`，验收时要求返回引用文件里的检查码 `DSH-SKILL-REF-01`。这同时检查发现、正文加载和相对引用，不要求新建 HTTP API。按第 7.2 节准备运行时，再走第 7.9 节的 Skills 路线；不用启动 HTTP 服务。

只有角色说明时怎样接。 可以在业务 bundle 的 patch 中新增一个具名 preset，把角色正文放到它的 persona 配置。以下示例面向已有 base + web 的 profile；它不复制标准 preset 的全部子插件，也不会自动读取任意 agents 目录。

```yaml
- insert:
    - id: team.component.reviewer-preset
      name: '@deepseek-ai/dsh-agent-preset'
      config:
        id: component-reviewer
        name: Component Reviewer
        description: Review component contracts from supplied material.
        plugins:
          - id: reviewer-persona
            name: '@deepseek-ai/dsh-persona'
            config:
              prefix: >-
                Review the component material supplied by the user.
                Separate verified facts from suggestions.
                Return missing contracts and concrete acceptance steps.
```

包要声明与宿主兼容的 agent-preset/persona peer；它们是宿主插件依赖，不是模型配置。安装后在设置的 Agent preset 选择 `component-reviewer`，**新建会话**并粘贴一张接入卡来验收。现有会话不会因为更换默认 preset 自动换角色；需要文件、skill 或其他工具时，在这个 preset 的 plugins 中显式加入所需插件，或以目标版本的标准 preset 完整子列表为基础调整。仅换角色不保证工具权限收紧。Agent preset 使用契约 [S09]

上述角色 patch 应放入自己的配置型 Bundle，包清单按第 7.9 节的角色路线修改；安装后选择对应 preset 并新建会话。persona 必须放在 Agent 的作用域中；直接把 `dsh-persona` 插在 Host 根会与全局 persona 冲突。普通角色配置仍复用 DSH 的模型与会话；完整外部 Agent 的接法见下一节。Persona 作用域实现 [S10]

5.4 ACP 完整 Agent

ACP 用于客户端与 Agent 交互，涉及初始化、会话、任务输入、过程通知、权限和取消；它适合交付拥有自己运行时的完整 Agent。ACP 协议概览 [S11]

DSH 的两个方向需要分别配置：

方向	DSH 入口	用法
其他客户端运行 DSH Agent	`dsh --profile acp`	DSH 提供 ACP 服务
DSH 委派任务给外部 ACP Agent	`@deepseek-ai/dsh-subagent-acp`	DSH 启动外部 Agent 并调用它

外部组件先实现并验证 ACP，再由 bundle 配置 provider 的 `command`、`args`、`env`、`providerName` 与权限策略；同时确认宿主具有 subagent 服务和可派发该 provider 的工具。只登记后端，不等于模型已经有调用入口。下面同时装配后端和 `component_agent` 委派工具；两者 provider 名必须一致，包还需声明匹配的 tool-subagent 依赖。

按第 7.9 节的 ACP 路线准备配置型 Bundle，将下面 patch 放入包中；安装前先确认外部 Agent 的启动命令可以完成 ACP 握手。以下是外部 ACP Agent 的装配结构。命令路径和参数是占位项，必须替换为该组件真实的 ACP 启动命令；bundle 要声明或确保宿主可解析匹配版本的 `dsh-subagent-acp`。

```yaml
- insert:
    - id: team.component.acp
      name: '@deepseek-ai/dsh-subagent-acp'
      config:
        providerName: team-component
        command: 'D:/tools/component-agent.exe'
        args: ['acp']
        permission: reject
        env: {}
    - id: team.component.acp-tool
      name: '@deepseek-ai/dsh-tool-subagent'
      config:
        provider: team-component
        toolName: component_agent
```

当前 DSH 的 ACP 子 Agent 后端每次任务启动新进程，使用外部 Agent 自己的模型、工具和会话；默认 `permission: reject`。它不提供父 Agent 的 persona、工具过滤等可选能力，每次 activation 也不能追加输入。因此接入前必须判断项目是否接受这种委派方式。ACP 协议能够表达多轮会话，不代表这个 DSH 后端暴露了全部多轮能力。DSH ACP 子 Agent 后端 [S12]

验收至少覆盖：握手、新建会话、任务执行、最终产物、权限拒绝、取消、子进程退出、失败与部分输出。ECAT 是否走 ACP，取决于它要保留完整自主执行，还是把评估能力交给 DSH 来编排。

调用验证：在新会话要求“使用 component_agent，让外部 Agent 对给定输入执行它支持的最小任务，等待完成并返回产物”。工具最初返回子任务 ID 不代表任务已经完成，还要核对完成通知、最终结果和外部进程退出。权限被拒绝是可预期的验收分支，不能为通过示例直接改成全放行。委派工具配置 [S13]

5.5 已有 CLI 或 SDK，直接复用

先调用已有能力，再决定宿主需要哪种包装。不要先为 CLI/SDK 新建 HTTP 层。

现有情况	最小接法	接下来做什么
CLI 已能被宿主 shell 工具调用	用 Skill 写固定命令、参数、输出和失败判断	按 5.3 交付资源，7.9 打包安装；验证真实命令结果
CLI 需要固定工具参数或宿主不直接用 shell	薄工具桥接，或一个调用 CLI 的 stdio MCP 包装	只映射参数并调用同一个 CLI；MCP 按 5.2 接，不新增 HTTP
SDK 与插件运行时兼容	插件导入已发布的库，调用公共函数	声明依赖；注册工具并映射输入输出；包结构与 7.9 通用步骤复用
SDK 语言与宿主不同	优先已有 CLI；没有时做最小 CLI/stdio MCP 包装	保持业务在 SDK 中；有远程或共享服务需求再评估 HTTP

CLI 路线可以复制 C.3 的资源包结构，把 Skill 正文替换为真实 CLI 的安装、命令、输入和输出说明；同时调整包名、描述和所需引用文件。使用者已有命令工具即可按说明执行，不必再做自定义业务插件。若自行执行子进程，使用固定可执行文件和参数数组，明确输出与非零退出码；不要拼接模型提供的 shell 字符串。

SDK 路线可参考 C.1 的包与工具注册结构，但把 HTTP client 调用换成 SDK 公共方法，并移除 baseUrl/tokenEnv 等无关配置；它不是直接运行 HTTP 示例。按 7.9 的通用命令打包准备好的目录，首次调用按 SDK 的真实输入输出验收。

AgentLineage 已有不启动服务的 SDK/CLI 路线；当前 DSH 插件选用 HTTP，所以保留该插件仍需 Core 服务。Learning 的调查预算、取消和经验发布属于它自身的业务需求，其他组件无需照抄。

6 Plugin 与 Bundle 的拆分、装配和发布

给最终 DSH 使用者交付可安装 bundle，内部先保持最少的插件。一个 bundle 只有一个 plugin 完全正常。 Plugin 是行为和生命周期的单位，Bundle 是安装与装配的单位，它们可以由同一个 npm 包交付。

先按第 6.1-6.4 节决定职责与包边界，再从第 6.5、6.6、6.7 节选择一种装配结构；第 6.8-6.11 节检查配置、依赖和交付内容。做完进入第 7 章安装验收，准备发布时再执行第 6.12-6.17 节。

6.1 Plugin 与 Bundle 不是二选一

先问“谁来安装、谁来组合”，再决定运行时插件、包边界和默认 Bundle。组件开发者可以复用 Plugin 入口；最终用户通常安装带默认装配的 Bundle，两者可以由同一个包提供。

使用者与需求	建议交付	使用者还要做什么
普通 DSH 用户，希望安装后能找到并配置能力	一个 bundle，内部可以只有一个插件	配服务、凭据和工作区，再按说明验证
其他插件开发者，希望把你的能力组合进自己的产品	可独立引用的 plugin 包；需要时同时提供默认 bundle	声明依赖，并在自己的 patch 中装配
已有 MCP 服务，只需配置连接	配置型 bundle，复用 DSH MCP client	启动服务，配置传输与鉴权
一组 skills、agents 或工作流资源	带资源及加载配置的 bundle；必要时附适配插件	核对资源发现、工具映射和执行环境
多个能力共同完成一个场景	组合 bundle，明确必需和可选插件	一次安装，按需启用可选能力

使用者与需求	建议交付	使用者还要做什么
多个独立场景只是同属一个团队	多个 bundle，由 profile 选择	按业务选择组合，不被迫装全家桶
HTTP/CLI/SDK 给非 DSH 产品使用	按原生方式发布公共组件；另交付 DSH bundle	各宿主消费同一个公共入口

按本文核对的 DSH 源码，插件安装命令会识别包清单中的 `dsh.bundle.patch`，将新装的 bundle 加入 profile 的组合列表。没有该声明的包可作为普通依赖安装，但不会自动成为 profile 配置层。只给用户一个 plugin 入口、让用户自己猜配置，不适合作为团队默认的最终交付。DSH 安装实现 [S14]

“发布一个插件”在日常交流中可以泛指整个交付物；在代码与交付卡中应写清：npm 包是什么、导出几个 plugin、是否带 bundle patch、要依赖哪些独立服务。

真实社区例子也采用这种组合：Awesome DSH Plugin 的收录说明要求交付物声明 dsh.bundle；dsh-market 自身的 package.json 同时导出 Plugin 入口并声明 Bundle patch。因此市场称它为“插件”，与工程上通过 Bundle 发布并不矛盾。Awesome 收录口径 [S15]、dsh-market 包清单 [S16]

具体决策顺序是：先定 Plugin 职责，再定同包或分包，最后决定向用户交付一个还是多个可安装 Bundle。单个 Plugin 也适合配默认 Bundle；不要为了使用 Bundle 而人为增加插件数量。

6.2 什么时候拆成多个 Plugin

先列出能力，再逐项判断。存在明确的独立使用需求才拆；暂时没有时，用一个插件内的模块和函数组织代码即可。

判断项	倾向拆分的例子	保持同插件的例子
是否需要独立启停	用户要查询 Memory，但不采集会话	search/read 共用配置且总是一起启用
生命周期是否不同	采集器持续订阅，查询工具只在调用时工作	几个短查询工具没有独立后台资源
配置和运行依赖是否不同	只读查询可用，动态评测另需模型与 runner	多个工具调用同一服务、使用相同身份
宿主作用域是否不同	Host 采集器与某个 preset 的角色提示	同一作用域的一组相关工具
是否有独立复用者	Lineage 工具既被联合分析包使用，也被其他产品使用	只是同一功能的解析、校验、格式化模块
失败与重启处理是否不同	采集源故障时仍希望查询已入库数据	操作必须一起成功或一起回滚
操作权限是否不同	检索与提交事件需要不同服务端身份	两个只读查询处于相同授权范围

权限差异可以作为拆分职责的理由，但**两个 Host plugin 仍可能运行在同一进程；拆插件不提供安全隔离。** 真正的权限边界要由服务端校验、独立凭据或进程身份、受管执行与必要的进程隔离实现。仅把写入工具标成“默认关闭”也不是授权校验。

同样，拆分后可以分别配置激活和依赖策略，但不能自动隔离同进程崩溃、内存耗尽或全局状态损坏。需要故障隔离时，把对应工作放到受管进程或独立服务。

拆完后反过来检查

每个插件应能回答四句话：

1. 我提供哪一组用户可理解的能力。

2. 我依赖什么服务、配置和权限。
3. 我拥有哪些注册、订阅、连接、定时器或子进程，停用时怎样释放。
4. 我关闭后哪些功能受影响，其余功能为什么仍然成立。

如果两个插件总要一起装、一起开、一起配置、一起重启，并且没有独立复用者，优先合回一个插件。多个插件相互传内部对象或彼此触发启动，通常也说明边界需要调整。

6.3 不按哪些方式拆分

- 不按 HTTP endpoint 拆：`search`、`browse`、`read` 可以是同一查询插件里的三个工具。
- 不按源代码目录拆：client、schema、parser、formatter 通常是库模块。
- 不按资源文件拆：几十个 skills 或角色文件可以共用一个 provider；需要不同装载策略时再拆。
- 不按程序语言拆：Python 服务可以由一个 Node 插件调用，语言差异不直接决定插件数。
- 不把 API、MCP、DSH 三种入口各写一份业务逻辑。它们消费同一内核，入口差异只保留必要的协议处理。
- 不把 UI 与核心强绑：需要 DSH 客户端视图时按其客户端机制提供，不能认为工具注册自动生成完整业务页面。
- 不为凑数量拆公共服务：只有多个贡献确实共享连接、缓存或生命周期时，才值得增加单独的服务插件。

HTTP 服务、Python wheel、工作流程序、Memory Release、DSH plugin 是不同层的交付物。它们之间可以有多对一关系，不要求数量相等。

6.4 多个 Plugin 放一个包还是多个包

这是第二个决定，与前面的运行时拆分分开做。

条件	同一个 npm 包，导出多个 plugin	多个 plugin 包，另做组合 bundle
维护与发布	同团队、同版本、一起发布	不同负责人或独立发布节奏
独立消费	主要被同一个产品使用	有多个产品分别使用其中一部分
安装依赖	依赖小且通常都需要	可选部分带较大依赖或不同运行环境
版本兼容	希望一次升级整组代码	需要单独升级、锁定或回滚某个模块
维护成本	少维护包和版本组合	承担跨包协议、发布顺序与兼容矩阵

首版通常采用同包多入口，先验证运行时边界；出现实际的独立消费者或发布需求后再拆包。若两个模块已经由不同项目维护，可以直接保留独立插件包，通过总 bundle 组合，不必先合仓。

同包的一个插件设为 `disabled: true`，仍可能把其代码和包依赖一起安装。若要求“用户不选就不下载这部分”，应单独发布为可选 bundle，由 profile 选择；仅关闭插件达不到这个效果。

6.5 单插件与 Bundle 同包

只有一个运行职责时采用此结构。下面展示需要哪些文件；首次实现可直接复制附录 C.1 的完整目录，再替换业务逻辑。不要只复制以下清单就运行，它还缺实际插件代码。示例版本号属于示例包：

```text
my-component-dsh/
  package.json
  cordis.patch.yml
  index.js                或构建后的 dist/plugin.js
  assets/                 有资源时提供
  README.md
```

```json
{
  "name": "@team/component-demo",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "exports": { ".": "./index.js" },
  "files": ["index.js", "cordis.patch.yml", "README.md"],
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } }
}
```

```yaml
- insert:
    - id: team.component.demo
      name: '@team/component-demo'
      config: {}
```

这是结构示例。实际依赖、peerDependencies、资源文件和配置要随实现补齐。`private: true` 阻止发布到 registry，但不阻止打 tarball；它不负责包文件的访问控制。需要发布到私有 registry 时也要移除该标记，再由 registry 配置控制可见性。面向团队正式交付时，补充显示名称、描述、许可允许分发的图标及其导出。

6.6 同包导出多个 Plugin

以下用 Memory 的查询与采集说明结构，不是已经实现的 EngramForge DSH 包。两个入口模块及公共桥接仍需实现、验证。

```text
memory-dsh/
  package.json
  cordis.patch.yml
  dist/
    query-plugin.js
    capture-plugin.js
    client.js             共享客户端库，不自动算第三个插件
  assets/skills/
  README.md
```

```json
{
  "name": "@team/memory-dsh",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "exports": {
    "./query": "./dist/query-plugin.js",
    "./capture": "./dist/capture-plugin.js"
  },
  "files": ["dist", "assets", "cordis.patch.yml", "README.md"],
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } }
}
```

```yaml
- insert:
    - id: team.memory.query
      name: '@team/memory-dsh/query'
      config: {}
    - id: team.memory.capture
      name: '@team/memory-dsh/capture'
      disabled: true
      config: {}
```

`exports` 提供可解析的模块入口；patch 插入两个插件实例。这里的空配置只展示装配结构，正式包需按各插件 schema 配置，并声明兼容的依赖/peer。查询插件负责自己的工具，采集插件负责自己的订阅与清理。共享 client 是实现复用；若没有独立生命周期需求，不必再做成插件。

需要开启采集时，在对应 profile 的 patch 中显式设置：

```yaml
- id: team.memory.capture
  disabled: false
  config:
    adapterId: team-dsh-host
```

`adapterId` 是拟议配置字段，不构成身份凭据；涉及远程服务或跨身份写入时，还需按实际访问边界验证身份和权限。profile 覆盖整份 `config`，启用时必须提供真实实现要求的完整字段。

6.7 多个 Plugin 包组成一个 Bundle

以 Insight 与 AgentLineage 的组合为例，可以由一个交付包引用两个独立插件。以下名称是拟议名称：

```text
session-analysis-bundle
  dependencies: insight-dsh-plugin、agentlineage-dsh-plugin
  cordis.patch.yml:
    insert insight 插件实例
    insert lineage 插件实例
```

例如，发布的总 bundle 清单与 patch 可以采用以下结构。这里的两个 `@team` 插件包、版本和 `baseUrl` 字段都是方案示例；实现后必须以实际包名、兼容版本和配置 schema 替换。

```json
{
  "name": "@team/session-analysis-bundle",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "files": ["cordis.patch.yml"],
  "dependencies": {
    "@team/insight-dsh-plugin": "0.1.0",
    "@team/lineage-dsh-plugin": "0.1.0"
  },
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } }
}
```

```yaml
- insert:
    - id: team.session.insight
      name: '@team/insight-dsh-plugin'
      config:
        baseUrl: http://127.0.0.1:21025
    - id: team.session.lineage
      name: '@team/lineage-dsh-plugin'
      config:
        baseUrl: http://127.0.0.1:8110
```

Bundle 的 `dependencies` 让模块可解析；patch 决定加载哪些实例、传什么配置。**只声明依赖不会自动启用插件。** 同理，安装了某个 bundle 依赖，不等于递归选择其 bundle patch；总 bundle 需要明确装配插件，或由 profile 明确选择各子 bundle。不要同时用两种路径重复注册相同实例。

本地验证时，先让两个依赖包以对应版本在目标 registry 可获得，或在验证用清单中使用可解析的本地包地址；正式发布前换为交付环境能取得的固定版本。仅写上两个尚未发布的 `@team` 名称会导致安装失败。

两个模块先保持独立仓库、进程和版本也可以。一体交付要额外维护兼容版本组合、统一标识和验收场景。YAML 列表位置不保证服务加载顺序，运行时依赖通过插件的 `inject` 等机制表达。

6.8 一个 Bundle 组合到什么程度

一个 bundle 尽量对应一个使用者能命名的能力或场景，例如“会话分析与证据” “Memory 接入” “工作流运行”。Profile 再组合这些场景。

层	决定什么	不应该承担什么
业务内核与公共入口	数据、算法、业务状态与协议	DSH 的实例 ID 和界面细节
Plugin	注册、作用域、宿主生命周期与能力调用	跨项目的安装发布策略
Bundle	插件组合、默认配置、兼容依赖、资源	用户个人凭据、实际项目路径
Profile	本次运行选哪些 bundle，部署覆盖参数	复制各组件的业务实现

默认只启用承诺交付且验收过的功能。可选模型调查、动态执行、采集或自动注入可以默认关闭，用户按目标场景配置。必需依赖缺失时给明确错误；可选功能失效时说明它不可用，不能把整组能力显示为正常。

插件之间有真正的服务依赖时，用 DSH/Cordis 支持的依赖声明来约束激活与释放。不要依靠 YAML 排列顺序，也不要在插件内偷偷启动同级插件。若两个插件只是消费同一个外部服务，可以分别调用公共接口，无须互相依赖。

6.9 发布前的决策与交付检查

五项轻量要求是默认验收。以下是准备分发 DSH 包时的检查参考，只核对实际交付的功能；独立开关、组合测试、服务部署和数据回退按触发条件补充，不全部作为首版门槛。

在组件接入卡记录涉及的决定：

1. 最终使用者是谁；公共组件、Plugin 入口和默认 Bundle 分别提供什么，可以同时交付。
2. 每个 plugin 的职责、依赖、作用域、默认启用状态和资源清理方式。
3. 为什么拆分；若不拆会影响哪个实际场景。
4. 同包多入口还是多包组合；谁维护版本与兼容；涉及升级时补恢复方式。
5. 哪些进程与数据在 DSH 外部署，由谁启动和维护。
6. 哪些是当前实现、首版接入和后续方案。

不要只审核一张结构图。交付时还要检查：

检查	通过标准
发布对象	区分 Python wheel、npm 包、DSH bundle 与业务数据 Release
包内容	入口、exports、patch、资源和引用存在，实际 tarball 内容可核对
兼容性	固定已验证的宿主/插件/服务版本组合，声明 peer 与协议范围
命名与展示	包名、实例 ID、工具/provider 名可区分，描述对应实际能力
默认范围	默认安装与启用的功能符合承诺，不擅自启动治理发布或自动采集
独立开关（有可选项时）	关闭可选插件后必需能力继续工作；重新启用不产生重复注册
组合测试（声明组合时）	同一插件被多个 bundle 引用时有明确装配来源，无意外重复实例
缺依赖	未装服务、缺凭据、版本不兼容可定位，不是静默缺工具
清理与回退（涉及资源/旧数据时）	清理自有资源，保留业务数据；真实涉及数据格式变更时说明回退条件
使用者验收	使用声明版本完成成功与常见失败；干净环境复跑可按分发场景增加

内部可先分发 tarball；要发布到包注册表，再按仓库与团队规范确定包名、可见范围、发布身份和许可证，并处理 `private: true`。完成代码不等于已发布；发布不等于市场已收录，收录也不等于 DSH 运行成功。完整流程见第 6.12-6.17 节。

EngramForge 内已有名为 Release/bundle 的治理与发布制品，Layr 也有程序目录发布。它们不因同名而成为 DSH bundle；DSH 的判定依据仍是 npm 清单与装配 patch。EngramForge 发行单元 [S17]、Layr 项目说明 [S18]

6.10 复制一个确实有两个插件的 Bundle

附录 C.4 和 `examples/http-skills-bundle/` 提供可打包的示例组合：HTTP 查询与 Skill 资源加载在同一个 npm 包，各有导出入口和实例 ID。它们共享版本，但可以分别启停；只关闭 Skill 后，HTTP 查询应继续可用。

决定	示例组合的具体结果
对外安装一个什么	@team/component-http-skills-demo
内部两个入口	./health 对应 http.js；./skills 对应 skills.js
每个插件负责什么	health 注册 HTTP 工具；skills 注册包内资源 provider
默认启用什么	health 开启，skills 关闭；需要方法指引时再开启
为什么不拆两个 npm 包	同团队同版本交付，没有独立发布要求
怎么验收拆分	开关 skills 不影响 component_health；重新启用不重复注册

该组合复用 C.1 的 HTTP 代码和 C.3 的 Skill 加载代码。它与两个独立示例包是**替代安装方案**，请用新的测试 profile 试验，不要把它们全部安装到同一 profile，否则会重复注册工具或 provider。`disabled` 关闭激活，不会减少已经安装的包文件。

6.11 三类依赖不要混在一起

写在哪里	表示什么	常见错误
dependencies	随包需要解析的普通库或独立组件插件包	以为列在这里就自动加载了插件
peerDependencies	与宿主共用并要求兼容的 DSH/Cordis 包	从源码复制 workspace:*，直接发布到树外项目
插件 inject / patch	inject 等待服务；patch 决定加载实例和配置	以为 npm 依赖可替代运行时服务，或靠 YAML 顺序等待

本手册的 JS 示例包可直接打包，不需要 TypeScript 构建。使用第三方库时补 dependencies；使用 TypeScript 时先运行项目构建，令 exports 指向构建后的 JS，并把产物放进 files。单独运行 `node index.js` 不能验证 DSH 插件：它需要宿主解析依赖、创建 ctx、注入服务并调用 apply。

从 DSH monorepo 复制代码时，把 workspace 协议改为团队验证过的真实版本；旧插件的版本豁免不等于兼容。新插件先使用本手册的包结构，避免无意复制整个 DSH 内部构建环境。

6.12 市场发现、包分发与 DSH 加载是三件事

市场页面常把可安装的扩展统称为 plugin。安装包内部可以包含 Bundle 配置、一个或多个 Plugin、Skills 和客户端资源；有些市场还收录原始 Skill、脚本或独立应用。不能只根据卡片上的“插件”标签判断安装方式。

环节	保存或处理什么	完成后说明什么
包分发：npm registry、GitHub Release、Git 仓库	固定版本的包、源码、依赖和校验信息	使用者有地址可以取得交付物
市场发现：社区目录、网站、市场插件	名称、简介、分类、来源、安装地址与检查记录	使用者能够搜索到交付物
DSH 安装与加载	把包加入指定 profile，选择 Bundle，合成 patch 并加载 Plugin	目标宿主可以解析并启用能力；真实任务还要验收

“Registry”可能指存放包的 npm registry，也可能指只提供元数据的社区目录。目录里的安装地址可以指向 npm、GitHub 或 Release，并不要求市场自己托管全部包。官方插件管理机制 [S19]

```text
维护者：构建并验证 → 发布 npm 包 / GitHub 固定版本或 Release
                                  ↓
发现：作者提交 / npm keywords / GitHub topics / 同步其他目录
                                  ↓
目录：筛选、去重、生成元数据 → 网站或 DSH 内的市场展示
                                  ↓
用户：按来源安装 → profile 选择 Bundle → 加载 Plugin → 真实调用
```

不需要先有统一官方市场，这条链也能运转。已核实的社区实现主要使用特定 topic、包关键词和维护者提交，或直接复用别人生成的目录；它们没有共同遵循一套“扫描所有仓库”的机制。

数量必须连同统计对象一起看。网站可以统计仓库、包、子目录、内置模块、资源或生态应用；同一交付物也会被多个网站重复收录。dsh.pub 展示内置模块与社区 Bundle，dsh.so 使用 artifacts 口径，其他目录可能只纳入符合安装约定的包。本次检索看到数千至上万条记录，但没有对全量记录去重或实测，不把它们汇总成“全生态成熟插件总量”。dsh.pub 口径 [S20]、dsh.so 说明 [S21]

同样要区分“已发现、结构检查通过、安装测试通过、启动测试通过、业务场景通过”。各站的检查等级由站方定义；市场收录和团队验收分别记录。

6.13 非官方市场、目录与发现工具一览

核对日期：2026-10-14。以下是本次查到的 15 个社区入口，不是全量名录，也不是推荐排名。一个入口可以兼有网站和 DSH 插件；不同入口可能共用数据。表内来源以维护者文档或站点公开实现为依据，未实际向这些平台提交项目或安装市场插件。

网页目录、社区平台与兼有安装功能的市场

入口	数据来源或实现	发布者如何进入
Awesome DSH Plugin [S15]	维护者收录列表，生成供其他网站消费的目录；要求交付物声明 dsh.bundle	按贡献指南提交 PR；它是重要的上游目录
dsh-market / dshmarket.com [S22]	从 Awesome 的 plugins.json 读取目录；另提供可安装到 DSH 的市场面板	向 Awesome 提交条目，不向市场应用仓库提交目录条目
dsh.pub [S23]	同步 DSH 源码目录、作者提交和 GitHub dsh-plugin topic；固定 commit 检查 Bundle	站点 Submit 提交公开仓库；子包要明确 package path；topic 发现另受扫描规则限制
dsh-ai.org [S24]	主要使用 Awesome plugins.json，另通过 GitHub topic 发现候选	先完成上游目录收录，再核对本站是否同步；独立提交政策以站点为准
DeepSeek.club Plugin Hub [S25]	有自己的插件列表、作者提交、附件和审核状态；上游自动采集来源未核实	从站点“发布插件”进入提交表单；见第 6.14 节
DSH Directory [S26]	公开 GitHub 仓库、dsh-plugin topic 和运营补充的候选；检查静态 Bundle	站点 Submit 链接到 GitHub issue 模板；也可由 topic 被发现
DSH Get [S27]	聚合 Awesome、hrhgit 插件管理器、OMDSH Hub 与 GitHub topic，并保留来源字段	在数据仓库提缺失或纠错 issue，或按官网入口操作；仓库快照比网站更新慢
dsh.so [S21]	汇总 GitHub/npm 元数据，提供站方定义的 L1-L5 检查状态；也包含生态应用	Submit GitHub URL；站方说明支持通过 dsh-plugin / dsh-pack topic 发现
DSHX [S28]	展示版本、兼容性、维护者与指标；本次未核实完整自动采集来源	页面有 Submit plugin；具体审核和同步规则待确认
dsh-plugin.net [S29]	GitHub 项目目录，区分 Native Bundle、桥接和相关项目；只对部分记录深入检查	使用站点 Submit / repository preflight；收录与运行验证分别标记
dshplugins.co [S30]	页面说明来自 npm 与 GitHub；按包结构复核	页面建议添加 dsh-plugin keyword；公开数据日期仍为 2026-08-15，先确认是否继续维护
dsh-plugin.shop [S31]	站点说明每日从 GitHub 采集；另提供 @ace-zone/dsh-market	有“上传插件”入口；本次未核实全部筛选规则和提交后流程

可安装的市场与会话内发现工具

项目	数据来源或实现	作者与用户需要知道什么
LivXue/dsh-plugin-shop [S32]	自动采集 npm keywords 与 GitHub topics 中的 dsh-plugin / deepseek-harness，检查后生成目录	补齐关键词、Bundle、许可和源码信息；未出现时查构建拒绝记录；与上表 dsh-plugin.shop 是不同项目
bradeGithub/DSH-Plugins-Marketplace [S33]	定时扫描 dsh-plugin topic，生成静态索引；还提供 Skills 目录与自己的类型适配流程	Bundle 交给宿主管理器；原始 Skill、脚本等可能走市场自有安装路径，不能套用同一验收结论
dsh-find-plugin [S34]	在 Agent 会话内搜索 GitHub dsh-plugin topic	属于搜索入口；检索到仓库后仍需确认安装形态，不是又一份独立托管包仓库

域名、仓库和 npm 包名要一起记录，避免把相似名称当成同一项目。网站中的“官方”“认证”“Verified”也应确认指的是作者身份、社区审核还是某项测试，不自动解释为 DeepSeek 官方认可。

团队首发不必逐站重复上传。先做好一个规范交付物和固定安装地址，再选择一个有明确流程的上游目录，以及与目标用户匹配的社区平台；随后检查下游同步。没有收录承诺的平台，不写保证上线时间。

6.14 DeepSeek.club 应怎样理解和提交

DeepSeek.club 自称第三方开源生态社区，包含 Plugin Hub、模型、应用与社区内容。本次读取了公开插件详情页和站点发布的前端代码：列表区分 DSH 与其他生态内容；提交表单支持名称、slug、摘要、说明、类别、版本、源码地址、网站、安装命令、作者、许可与附件。表单有待审核和已发布状态。这能证明它提供独立提交与分发流程，不能证明后台如何抓取全部条目，也不能证明所有账号适用相同审核路径。社区主页 [S35]、提交入口 [S36]、公开提交表单实现 [S37]、列表实现 [S38]

建议按以下步骤发布；界面字段和文件限制仍以提交当天的页面为准：

1. 先完成自己的 npm/GitHub 发行和第 7 章的安装验收，保留版本、commit、安装地址及结果。若选择由该站托管附件，也先验证附件内容和安装方法。
2. 进入 Plugin Hub 的“发布插件”，按站点要求登录，选择适合的 DSH 类别，填写用途、版本、支持环境、源码和安装命令。
3. 按需提供演示、说明和附件。原始 Skill ZIP、npm pack 的 tgz、普通源码 ZIP 是不同交付物；在说明中写出真实安装步骤，不能假定它们都能直接 dsh plugin add。
4. 提交后查看站点返回的状态；待审核时跟踪“我的插件”，只有详情页公开可访问才记为已收录。发布新版本后检查安装命令、附件和详情版本是否同步。

公开详情中确实存在按 Skill ZIP 分发的条目，因此“该站有插件页”不足以证明其内容是原生 DSH Bundle。Skill 条目示例 [S39]

目前未找到可证明它直接同步 Awesome 或某个统一官方 registry 的公开依据；文档不作此推断。本次只读核实了页面和前端提交结构，没有登录、上传或走通审核，也未验证自动采集频率。带 hash 的前端资源 URL 会随站点发布变化，复核时从提交页面重新定位。

6.15 从本地交付物发布到 npm 或 GitHub

先完成第 7 章的本地打包与真实安装，再选择分发渠道。市场收录引用同一份交付物；不要为了每个市场复制一套业务实现。

发布路线	交付物	发布后必须检查
npm	固定 name/version 的已构建包	registry 返回的版本、repository 与完整性信息正确；干净 profile 可按该版本安装
GitHub Release	对应源码 commit 的版本 tag，加已验证的 tgz 与 SHA256	下载到的附件与验收包一致；公开安装地址可以访问
GitHub Git spec	固定 commit 的可安装包目录	manifest、patch、入口和资源都在该 commit 中；子目录写明；安装不依赖作者机器

发布前补齐元数据。 在业务包中设置自己有权限使用的 name/scope、version、description、license、repository、files、exports、兼容依赖与 dsh.bundle.patch。为自动发现补 npm keywords 中的 dsh-plugin / deepseek-harness，以及 GitHub 仓库相应 topics。关键词有助于被发现，不代替 Bundle 配置或市场检查。

下面只是要补入现有 package.json 的字段片段；保留原有的入口、依赖、dsh.client 和资源配置。替换 scope、仓库和实际许可后才能使用。示例包保留 private: true；业务包发布到公共或私有 registry 前都要移除该标记，访问权限由发布配置和 registry 控制。

```json
{
  "name": "@your-scope/dsh-cpx",
  "version": "0.1.0-rc.1",
  "description": "Capture model requests and responses for DSH debugging.",
  "license": "MIT",
  "repository": {
    "type": "git",
    "url": "git+https://github.com/your-org/your-repo.git"
  },
  "keywords": ["dsh-plugin", "deepseek-harness", "observability"],
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } }
}
```

许可示意不替代项目的实际授权；monorepo 同时声明 repository.directory，并在 README 写清 DSH 包所在目录。部分自动采集器只检查仓库根目录；子包可以优先发 npm，或走支持 package path 的显式提交，不保证打一个 topic 就能找到所有子包。

npm 路线。 在发行目录完成构建和测试，使用 npm pack --dry-run 检查清单，再打包并按第 7 章验证该 tgz。下面命令展示发布已验证 tarball 的顺序；变量必须替换为真实值，发布前还需具备 scope 权限，并完成 registry 要求的认证。它们不是本次已执行的发布记录。

```powershell
$releaseTarball = 'D:\releases\your-scope-dsh-cpx-0.1.0-rc.1.tgz'
$releaseSpec = '@your-scope/dsh-cpx@0.1.0-rc.1'
$releaseRegistry = 'https://registry.npmjs.org/'
npm.cmd publish $releaseTarball --dry-run --access public --tag next --registry $releaseRegistry
if ($LASTEXITCODE -ne 0) { throw '发布预演失败，先修正再继续' }
```

核对预演的包名、版本、文件清单和目标 registry 后，单独执行正式发布：

```powershell
npm.cmd publish $releaseTarball --access public --tag next --registry $releaseRegistry
if ($LASTEXITCODE -ne 0) { throw '发布失败，先核对 registry 状态' }
npm.cmd view $releaseSpec version repository dist.integrity --registry $releaseRegistry
```

预发布显式使用 next；稳定版本采用团队确认的版本和标签。修复已发布内容必须升版本，不能覆盖同一 name/version。发布后的安装验收使用固定版本；有些目录只索引稳定版本或 latest，预发布未出现时先查规则。npm publish 规则 [S40]

GitHub 路线。 让 tag 对应已验证的源码 commit，并在 Release 中附上同一个 tgz、SHA256、变更说明、支持版本和安装命令。若直接从 Git 安装，先验证该 commit 的包目录确实可安装；只有 TypeScript 源码、缺失 dist、依赖 workspace:* 或需要额外构建的仓库，不能当作已经完成分发。GitHub 默认源码 ZIP 与 npm tarball 的结构不同。

发布后复用第 7 章的独立环境，创建新的验收 profile，从公开来源重新安装一次，覆盖首次调用和失败恢复。确认自己测试的是发布后的包，而非缓存中的工作目录。把验证记录与发行版本关联，之后再提交市场。

6.16 从已发布的包到市场收录

准备一份收录材料。 至少包含：项目名与一句用途、目标用户、类别、源码仓库及子包路径、固定版本或 commit、规范安装地址、支持的 DSH/Node/系统版本、外部服务要求、默认启用范围、首次使用例子、截图或脱敏产物、维护者和反馈入口。对 CPX 要写清支持的模型协议、捕获范围和记录位置；对组合包要写清包含哪些组件及哪些默认关闭。

渠道	操作	什么算完成
Awesome → dsh-market 等下游	按当前贡献指南修改目录并提交 PR；等待评审后检查生成目录和下游展示	PR 合入、目录可查、卡片安装地址正确；三者分别核对
dsh.pub	从 Submit 提交公开仓库，子包明确 path；跟踪提交状态和检查结果	目录出现对应 commit 与安装入口；topic 自动发现另按站方规则
topic / keyword 自动发现的市场	加 GitHub topics 和 npm keywords，确保 manifest、源码、许可和构建产物齐全	扫描后能查到目标包；若被拒绝，根据该市场报告修复，不把“加标签”记为已收录
DeepSeek.club	按第 6.14 节填写详情和安装方式，按需上传附件，跟踪审核	已发布详情可访问，版本、附件和安装命令一致
其他目录	使用第 6.13 节列出的 Submit / issue 入口，按本平台规则提交	保留结果链接与核对日期；没有公开规则的先确认，不推测自动同步

Awesome 的具体修改位置、检查要求和接受范围，以其贡献指南 [S41]为准。dsh.pub 的自动提交、固定 commit 检查与 topic 同步，以其维护者说明 [S23]为准。两条流程独立，向其中一处提交不代表全部市场同时收录。

收录后再从市场卡片取得安装地址，在干净的测试 profile 完成安装和真实调用。升级时先发布新版本，再更新需要手动维护的条目和附件，等待自动目录刷新；最后确认卡片没有仍指向旧包或浮动分支。

6.17 发布与收录的记录方式

状态	必须保留的证据
本地可打包	实际 tarball、文件清单、SHA256
已发布	npm 固定版本或 GitHub Release/commit、公开下载位置
已提交市场	PR、issue 或站内提交状态；此时不写已收录
已收录	可访问的详情页、展示版本、安装地址、核对日期
安装与使用已验收	目标 DSH/Node/系统、干净 profile、真实调用和失败恢复记录
已维护	负责人、更新渠道、上一版本与回滚说明、失效条目的纠正记录

选择公开市场之前，先确定组件是否面向外部用户。仅团队内部使用的组件可以通过私有 registry、固定 tarball 和团队 profile 分发。公开发布时同时提供独立安装与组合安装说明；市场有无流量，不改变插件职责与生命周期的划分。

7 从打包到 DSH 启动

按组件状态选择路径：**已有 npm/Git/压缩包发布地址，按第 7.1 节安装；开发 HTTP 组件，按第 7.2-7.8 节执行；开发其他组件，按第 7.2 节准备后转第 7.9 节。** 第 7.2 节以后使用 Windows PowerShell；Mac/Linux 保留相同 CLI 语义，调整路径和环境变量写法。

7.1 DSH 原生安装：直接从发布源获取

DSH 已提供原生安装命令，不要求安装第三方市场，也不要求组件团队再开发安装 CLI：

```text
dsh plugin --profile <profile名称> add <安装地址>
```

本手册基线要求显式传入 `--profile`，且它必须写在 `add` 之前；不要把 `dsh plugin add xxx` 当作完整命令。DSH 在目标 profile 内调用 pnpm 下载和安装包，并把声明了 `dsh.bundle.patch` 的依赖加入该 profile 的 Bundle 列表。因此，同一个安装命令可以装单插件 Bundle，也可以装多插件 Bundle。第三方市场负责帮助用户找到安装地址，不是原生安装的前置依赖。官方 CLI 行为参考 [S42]

第一步：确认运行时和目标 Profile。 使用已经安装、与目标组件兼容的 DSH，运行 `dsh --version`；npm CLI 路径还需要 pnpm 在 PATH 上，可用 `pnpm.cmd --version` 确认。没有全局 `dsh` 命令时，按第 7.2 节用固定版本的 npx 或源码入口执行相同参数。Desktop 使用其内置插件页或该安装提供的命令，普通 npm CLI 不能管理 Desktop 的保留 profile。不同入口要对应同一套 DSH_HOME 和运行时。

第二步：选择一种发布源。 下表都是安装地址写法，包名、版本、仓库、commit、域名及本地路径须替换成作者提供的真实值；不表示这些示例包已经发布。

来源	原生命令示例	是否先手动下载
npm 或已配置的 registry	`dsh plugin --profile web add --save-exact '@your-team/component@0.1.0'`	不需要，由包管理器下载
GitHub 固定 commit	`dsh plugin --profile web add 'github:your-org/component#COMMIT_SHA'`	不需要；目标仓库须包含可安装包
远程 tarball	`dsh plugin --profile web add 'https://example.com/releases/component-0.1.0.tgz'`	不需要；地址须直接返回包文件
本地 tarball	`dsh plugin --profile web add 'D:\packages\component-0.1.0.tgz'`	先取得该文件，适合内部分发
本地目录	`dsh plugin --profile web add 'D:\work\component'`	用于本地开发；目录须有完整包清单和入口

GitHub Release 页面本身、普通源码 ZIP、任意仓库目录都不能直接等同于可安装 Bundle。优先发布带完整构建产物的 npm 包或 `npm pack` 生成的 `.tgz`；Git 安装若依赖 `prepare`，须按实际 pnpm 提示处理构建脚本。私有源先配置 registry、认证和访问权限；认证信息不放在公开安装命令中。离线分发 tarball 仍可能需要下载依赖，离线交付须另行准备完整依赖和运行时。

第三步：安装到实际使用的 Profile，再配置和启动。 下面以随附 `web` profile 为例；已有业务 profile 时，三条命令使用同一个名称。首次管理 `web` 时会使用随附模板初始化它。

```powershell
# 替换为已经发布且与当前 DSH 兼容的真实包名和版本
dsh plugin --profile web add --save-exact '@your-team/component@0.1.0'
if ($LASTEXITCODE -ne 0) { throw '安装失败，请先处理命令输出' }

# 按组件 README 配置服务地址和凭据后检查合成结果
dsh --profile web --dump-config
if ($LASTEXITCODE -ne 0) { throw '配置检查失败' }

# 已有该 Profile 进程时，先退出旧进程再启动
dsh --profile web
```

安装到一个新的自定义名称只会得到 base，不会自动得到 Web。需要独立 Web profile 时，按第 7.4 节从 Web 模板创建，再安装。`--from-default-profile` 只用于创建新 profile：本手册固定快照拒绝覆盖，但 2026-10-15 的上游文档已将其定义为重置目标目录，因此已有 profile 不重复传该参数。上游 Profile 创建与重置说明 [S43]

安装命令更新的是 Profile 的依赖与 Bundle 列表，不改组件包里的 patch；四类文件的位置和职责见第 1.1 节。检查目标 profile 的 `package.json`：`dependencies` 有目标包，`dsh.profile.bundles` 有该 Bundle，合成配置有预期实例。CLI 可以安装普通 npm 依赖，但缺少 `dsh.bundle.patch` 时不会自动作为 Bundle 启用；包下载成功不代表工具已可用。按组件 README 启动外部服务，再完成真实调用。更新已有包后重启，以加载新的代码。

也可以直接使用内置 Web 界面。 打开侧栏“插件” → “添加插件”，输入同一个包名、Git 地址、压缩包地址或本地绝对路径 → 检查来源及版本 → 安装 → 按界面提示启用。这个页面是 DSH 自带的管理入口，不需要先装社区市场插件。本地路径由运行 DSH 的 Host 读取，不能填另一台浏览器电脑上的路径。界面安装完成后仍须配置组件并确认激活状态。内置插件页 [S44]

官方可选组件也使用这套安装能力。例如官方 CLI 文档给出了 `@deepseek-ai/dsh-subagent-codex` 和 `@deepseek-ai/dsh-subagent-claude-code` 的 `add` 命令；实际交付应选与目标 DSH 兼容的版本。内置插件页还可展示官方随附或按需安装条目。“DSH 官方安装器”“npm 官方 registry”“第三方插件市场”分别指安装入口、包分发源、发现目录，不能混为一个服务。原生插件管理器 [S19]

组件团队需要交付什么： 可安装的 Bundle 包、固定版本地址、兼容范围、配置说明、外部服务启动方式和首次调用指引。服务部署或环境检测可以另提供脚本；正常的 Bundle 下载与 Profile 安装直接复用 DSH 原生命令。市场收录按第 6.16 节作为独立步骤处理。

7.2 准备同一个运行时与独立接入目录

先在终端 B 完成公共准备，再从“已发布运行时”和“源码运行时”中选择一条。两条路线都使用同一组目录变量。HTTP 路线另开终端 A 运行服务；其他路线的外部依赖按第 7.9 节准备。

先准备满足目标 DSH engines 的 Node.js 和可从 PATH 调用的 pnpm。本次源码要求 Node `^22.19.0 || >=24.0.0`，packageManager 为 pnpm 11.28.5。缺少命令或版本不符时，按团队安装说明准备后再继续。Windows 可使用 `npm.cmd`、`npx.cmd`、`pnpm.cmd`，无须修改整台机器的 PowerShell 执行策略。

公共准备：两条路线都执行。 先解压配套文件，将 `$componentSource` 改成其中包含 `examples` 的目录。下面创建独立工作目录和 DSH_HOME，不复用现有业务 Profile。

```powershell
$ErrorActionPreference = 'Stop'
Get-Command node,npm.cmd,pnpm.cmd -ErrorAction Stop | Out-Null
node --version
npm.cmd --version
pnpm.cmd --version
$componentSource = 'D:\work\AgentLineage\docs\componentization'
if (-not (Test-Path -LiteralPath (Join-Path $componentSource 'examples'))) {
    throw '请把 componentSource 改为包含 examples 的手册目录'
}
$componentRoot = Join-Path $env:TEMP ('dsh-component-' + (Get-Date -Format 'yyyyMMdd-HHmmss-fff'))
New-Item -ItemType Directory -Path $componentRoot | Out-Null
$componentWorkspace = Join-Path $componentRoot 'workspace'
New-Item -ItemType Directory -Path $componentWorkspace | Out-Null
$env:DSH_HOME = Join-Path $componentRoot 'dsh-home'
$env:DSH_AGENTS_HOME = Join-Path $componentRoot 'agents-home'
$env:DSH_TOOLS_MODE = 'native'
$componentDshVersion = '0.2.1-alpha.2'
$componentRoot
```

路线 A：使用已发布的匹配版本。 以下函数只是固定运行时的 PowerShell 调用方式，不是需要另行发布的安装 CLI。

```powershell
function Invoke-ComponentDsh {
    & npx.cmd --yes "@deepseek-ai/dsh@$componentDshVersion" @args
    if ($LASTEXITCODE -ne 0) { throw "DSH failed: $LASTEXITCODE" }
}
Invoke-ComponentDsh --version
```

本次基线是源码版本 `0.2.1-alpha.2`，不保证每个镜像都有该版本。首次 npx 会下载运行时；版本不存在、权限或网络错误时，按团队登记的发布源取得匹配运行时，或选择下面的源码路线。不要改为 latest 后沿用旧兼容结论。DSH 运行入口 [S45]

路线 B：使用源码 checkout。 保留公共准备的变量，替换下面仓库路径；已有匹配构建产物时可跳过 install/build。没有构建产物时须完成两步，每步失败都停下处理。构建可能需要原生工具链，按仓库开发说明准备。

```powershell
Set-Location 'D:\work\deepseek-harness'
pnpm.cmd install --frozen-lockfile
if ($LASTEXITCODE -ne 0) { throw 'DSH dependencies failed' }
pnpm.cmd run build
if ($LASTEXITCODE -ne 0) { throw 'DSH build failed' }
$componentDshRepo = (Get-Location).Path
function Invoke-ComponentDsh {
    & pnpm.cmd --dir $componentDshRepo dsh @args
    if ($LASTEXITCODE -ne 0) { throw "DSH failed: $LASTEXITCODE" }
}
Invoke-ComponentDsh --version
```

确认版本后，后续只使用选定的 `Invoke-ComponentDsh`。源码入口用 `--dir` 固定仓库；Web 中仍须显式选择 `$componentWorkspace`。终端 A、B 不共享 PowerShell 变量；B 在后续步骤保持打开。若关闭 B，应恢复原来的目录变量、DSH_HOME 和同一启动函数，不能重新生成目录后误以为仍在操作原 Profile。

7.3 启动独立服务并打包

本节是 HTTP 路线的完整示例，只有选择该路线才启动此服务。CLI/SDK、stdio MCP 和资源路线按 5.5、7.9 继续，不执行这里的 HTTP 服务步骤。

终端 A 切换到本手册目录，启动 C.1 配套测试服务；它只监听回环地址，**不是 MCP Server**。首轮不设置 COMPONENT_TOKEN，若已有该变量，先开一个不带它的验证终端，或按服务鉴权配置插件。

```powershell
# 改为自己的手册目录；终端 A 不会继承 B 的 componentSource 变量
Set-Location 'D:\work\AgentLineage\docs\componentization'
Remove-Item Env:COMPONENT_TOKEN -ErrorAction SilentlyContinue
$env:DEMO_PORT = '8120'
$env:DEMO_MODE = 'ok'
node .\examples\fixtures\health-server.mjs
```

看到 `Ready: http://127.0.0.1:8120/health (ok)` 后保持终端 A 运行。若端口被占用，选择另一个空闲端口，并同步替换本章的 URL 和插件 baseUrl。

终端 B 先验证服务，再复制完整包文件打包：

```powershell
Invoke-RestMethod 'http://127.0.0.1:8120/health'
Copy-Item -LiteralPath (Join-Path $componentSource 'examples\http-bundle') -Destination (Join-Path $componen
  tRoot 'plugin') -Recurse
Set-Location (Join-Path $componentRoot 'plugin')
node --check index.js
if ($LASTEXITCODE -ne 0) { throw 'Plugin syntax check failed' }
npm.cmd pack --dry-run --ignore-scripts
if ($LASTEXITCODE -ne 0) { throw 'Package preview failed' }
$componentPack = npm.cmd pack --ignore-scripts --json | ConvertFrom-Json
if ($LASTEXITCODE -ne 0) { throw 'npm pack failed' }
$componentTarball = Join-Path (Get-Location).Path $componentPack[0].filename
Get-FileHash -LiteralPath $componentTarball -Algorithm SHA256
```

预期服务返回 `status: ok`；包包含 index.js、cordis.patch.yml、package.json、README 和 locale。`npm pack` 不需要先启动 DSH；本示例包没有构建脚本，不能把 `--ignore-scripts` 当成所有 TypeScript 项目的构建替代。实际 tarball 名用命令输出，不手猜 scoped 包的文件名。

7.4 创建 Profile 并安装

继续在终端 B 执行。先从随附 Web 模板创建 profile，再安装刚才的 tarball。

```powershell
$componentProfileDir = Join-Path $env:DSH_HOME 'profiles\component-check'
if (Test-Path -LiteralPath $componentProfileDir) {
    throw '目标 Profile 已存在；复用时跳过创建命令，或选择新名称'
}
Invoke-ComponentDsh --profile component-check --from-default-profile web --dump-config
Invoke-ComponentDsh plugin --profile component-check add $componentTarball
Get-Content -LiteralPath (Join-Path $componentProfileDir 'package.json')
```

创建区块只执行一次。复用已有 Profile 时，先确认名称和路径，再单独执行安装命令；不再运行 `--from-default-profile web`。重开终端后必须恢复同一个 DSH_HOME 与 launcher 函数，否则会操作另一份 profile。`--profile` 必须写在 plugin 的 add 之前。CLI 选项位置 [S46]

profile package.json 中应保留以下装配关系，其他字段由 DSH 管理；这是检查示意，不是覆盖文件的命令。

```json
{
  "dsh": {
    "profile": {
      "bundles": [
        "@deepseek-ai/dsh-base",
        "@deepseek-ai/dsh-web-app",
        "@team/component-demo"
      ]
    }
  }
}
```

同时检查 dependencies 中有 @team/component-demo。安装成功但 bundles 没有它，先查包是否声明 dsh.bundle.patch、是否在正确 profile 操作；不要通过反复插入同名插件补救。出现 peer 不兼容或包管理失败时，保留错误并核对版本与发布源。

7.5 配置地址并检查合成结果

下面只写入本章刚创建的独立验证 profile。已有业务 profile 应编辑原文件的对应行，保留其他配置。

```powershell
@'
- id: team.component.demo
  config:
    baseUrl: http://127.0.0.1:8120
'@ | Set-Content -LiteralPath (Join-Path $componentProfileDir 'cordis.patch.yml') -Encoding utf8
Invoke-ComponentDsh --profile component-check --dump-config
```

合成结果应有 `id: team.component.demo`、模块 `@team/component-demo`、正确的 baseUrl，且未禁用。`--dump-config` 不启动插件，也不证明服务连通。默认 profile 自带 tools 等宿主服务，不要在组件包里重复创建一套。Profile 与合成规则 [S47]

本 HTTP 示例包不需要 token；**不要照抄一个没有值的 tokenEnv**。接真实服务时，配置 `tokenEnv: COMPONENT_TOKEN` 并给启动进程提供该环境变量。tokenEnv 是变量名，不是 token 本身。

patch 替换目标行的整个 config，不做递归合并；覆盖时保留全部必需字段。优先级为 bundle 列表 → profile patch → home patch → 本次 --patch。CLI、Web 插件设置可能修改不同配置层，最终以 dump 与实际激活结果为准。

7.6 启动、选择模型并完成首次调用

静态安装和配置检查不需要模型，但 Agent 真正调用工具需要可用模型。用团队认可的凭据渠道给终端 B 提供 DEEPSEEK_API_KEY；需要手工输入时可使用下面的本进程赋值，不把 key 写入示例文件。

```powershell
$componentKey = Read-Host '输入本次测试的 API Key' -AsSecureString
$env:DEEPSEEK_API_KEY = [System.Net.NetworkCredential]::new('', $componentKey).Password
Set-Location $componentWorkspace
Invoke-ComponentDsh --profile component-check --no-open
```

采用其他认证方式时，跳过上述 key 输入，按目标运行时的模型设置操作。当前 DeepSeek 适配器使用 Messages 协议；自定义 DEEPSEEK_BASE_URL 必须是兼容 Messages 的根地址，不能把仅支持 Chat Completions 的地址当作通用替换。模型名称、服务权限按实际账号确认。模型配置与端点 [S48]

1. 打开终端打印的完整启动 URL。该 URL 包含本次进程的认证信息；只打开裸端口不一定能建立会话。
2. 在 Web 中选择 `$componentWorkspace`，不要假设终端 cwd 已自动成为选中的 Web 工作区。
3. 在“设置 → 模型”确认 provider、模型和凭据可用；选择标准 preset，新建会话。
4. 在插件/配置页面确认示例实例没有激活错误，然后发出下面的请求。

```text
请调用 component_health 一次，不要用 shell 或自行发 HTTP 请求代替。
报告工具返回的原始 status；如果工具不可用或调用失败，直接说明。
```

通过条件必须同时满足：会话中能看到真实的 component_health 调用、返回 `{"status":"ok"}`、终端 A 收到对应 GET /health 请求。只看到模型文字“服务正常”不算通过。服务日志用于交叉核对，记录时不要夹带 key 或带认证信息的启动 URL。

没有凭据或模型不可用时，停在“已安装/已检查配置”，验收表的“真实任务”仍填未验证。Web 能启动、插件可见、模型能聊天、目标工具成功是四个不同检查点。Web 启动与模型入口 [S49]

7.7 验证失败、可选开关与退出

每项从正常状态开始，记录结果后恢复。切换服务模式时，在终端 A 按 Ctrl+C 停止服务，设置 `$env:DEMO_MODE = 'invalid'` 等值，再只运行 `node .\examples\fixtures\health-server.mjs`；不要重跑第 7.3 节把模式重置为 ok 的整段初始化。恢复时设置 `$env:DEMO_MODE = 'ok'` 并重启服务。

验收项	怎么做	应观察什么
停服	终端 A Ctrl+C，再调用工具	工具失败，不能返回缓存的 ok
服务恢复	A 按原命令重启，再调用	同一地址恢复可用，出现新的请求记录
非正常状态	A 停止后设置 DEMO_MODE=degraded 再启动	返回 degraded，模型不得改写成 ok
非法响应	A 停服后改 DEMO_MODE=invalid，再启动	接口虽返回 200，插件仍报告 invalid health response
超时	A 停服后改 DEMO_MODE=slow，再启动	五秒左右取消请求，不能等到慢结果后当作成功
禁用	停止 B；在该实例保留完整 config 并加 disabled: true，再启动和新建会话	component_health 不可调用；恢复 false 后重新可用
退出	A、B 分别 Ctrl+C	各自进程停止；DSH 退出不会替你关闭另一个终端里的服务

可选 Skill 的开关验证使用 C.4 的新 profile。安装、激活和调用要分开记录；失败时按第 11 章定位服务、包、配置、作用域或模型问题，不连续重复安装。

7.8 换成业务组件并交付他人

从示例包改什么	改完怎样证明
包名、插件模块入口、实例 ID、工具名	package exports、patch name、调用名一致，不与其他包冲突
工具输入输出与 SDK/CLI/MCP/HTTP 调用	有真实业务样例和常见错误；慢调用再补超时/中断，参数不拼接任意 shell
配置与资源	仅要求实际依赖的配置；涉及地址/凭据时由使用者配置；随包资源和引用完整
依赖与版本	分发包不带未处理的 workspace:*；记录已验证宿主及实际依赖版本
名称、描述、图标和 README	使用者能识别能力，并按完整步骤安装，不依赖作者的 cwd
升级	增加版本；用新 tarball 安装，重启 DSH 后核实实际包版本

交付版本明确的 npm/Git/远程 tarball 地址或内部 tarball 文件，提供必要配置；SHA256 按发布流程或团队需要附上，并非所有内部组件的门槛；接收者按第 7.1 节直接使用 DSH 原生安装。无需先进入第三方市场，也无需为每个组件再提供安装 CLI。发布到团队注册表时，确定包 scope、可见范围、发布身份和许可证，处理 private: true。示例包名不能原样当作已存在的注册表包。

公开发布与市场收录继续按第 6.15-6.17 节操作；登记“已发布、已提交、已收录、已验收”四种状态。

需要升级已有安装时，保留上一个版本的包、配置与结果；替换后重启以加载新 JS，失败时恢复前一版本。只有涉及数据库迁移才补数据回退方案，首版无需为不存在的旧数据设计迁移。DSH 的包管理不负责回滚业务数据；组件不得把卸载当成默认删数据的操作。交付结论写清已验证的层级、未决依赖与故障处理，填写附录 B。

7.9 其他路线的打包和验收

这些路线及 5.5 的 CLI/SDK 包共用第 7.2 节的运行时与目录准备，但不照抄 HTTP 的服务地址、实例 ID 和 health 验收。先选下表的一行，准备对应目录和真实依赖。本示例使用新的 Profile 防止冲突，不把新建 Profile 当成每次交付的统一门槛。C.4 与 C.1/C.3 会注册相同工具或 provider，不能一起装入同一 Profile。

路线	起点和需要修改的内容	验收时做什么
MCP	复制 `examples/mcp-bundle`；按 5.2 选择现有 HTTP 或本地 stdio，替换完整 config 与实例 ID	调用真实工具并测常见失败；慢调用才补中断/超时，不要求 HTTP 服务
CLI / SDK	按 5.5 准备资源包、工具桥接或库适配目录，替换包名与业务入口	在目标宿主调用同一 CLI/SDK；验证成功与常见失败
Skills	复制 `examples/skills-bundle`；首轮保留完整 assets 和 provider 配置	新标准会话加载 component-check，读取 references/checklist.md，返回 DSH-SKILL-REF-01
HTTP + Skills	复制 `examples/http-skills-bundle`；启动 C.5 服务，按 C.4 启用资源行	先验 health，再验 Skill；关闭 Skill 后 health 仍可调用

路线	起点和需要修改的内容	验收时做什么
角色 preset	以 MCP 配置包为外壳，按下文替换清单和第 5.3 节 patch	选择 component-reviewer 后新建会话，提交接入卡，检查角色要求与所需工具
外部 ACP Agent	以 MCP 配置包为外壳，按下文替换清单和第 5.4 节 patch；换成真实外部启动命令	调用 component_agent，核对完成通知、产物、取消和外部进程退出

角色和 ACP 的配置包怎样准备。 复制 MCP 包到自己的新目录，修改 name、description、locale；保留 files 中的 patch/README/locale 以及 `dsh.bundle.patch`。将整个 `cordis.patch.yml` 替换为第 5.3 或 5.4 节对应 YAML，不把旧 MCP 行留在里面；移除原来的 MCP peer，并按路线加入以下 peer。这里沿用本手册版本，目标版本不同须重新核对兼容性。

路线	package.json 中的 peerDependencies
角色 preset	`@deepseek-ai/dsh-agent-preset` 与 `@deepseek-ai/dsh-persona`，均为 `0.2.1-alpha.2`
ACP	`@deepseek-ai/dsh-subagent-acp` 与 `@deepseek-ai/dsh-tool-subagent`，均为 `0.2.1-alpha.2`

这两类包装配已有 DSH 插件，不需要额外编写 index.js。目标 Host 仍须提供相应运行服务，ACP 外部程序及其模型凭据须另行安装。它们是配置指引，不表示已经提供了可运行的业务 Agent。

打包安装以 Skills 为例。 在第 7.2 节的终端 B 执行，其他路线把 `$routeSource` 改为已经准备好的包目录，把 `$routeProfile` 改成未使用过的名称。运行命令前完成上表的配置修改。

```powershell
$routeSource = Join-Path $componentSource 'examples\skills-bundle'
$routeProfile = 'component-skills-check'
$routeWork = Join-Path $componentRoot $routeProfile
$routeProfileDir = Join-Path $env:DSH_HOME ('profiles\' + $routeProfile)
if ((Test-Path -LiteralPath $routeWork) -or (Test-Path -LiteralPath $routeProfileDir)) {
    throw '请换一个未使用的验证名称，或明确复用原目录后分步执行'
}
Copy-Item -LiteralPath $routeSource -Destination $routeWork -Recurse
Set-Location $routeWork
Get-ChildItem -LiteralPath $routeWork -File -Filter '*.js' | ForEach-Object {
    node --check $_.FullName
    if ($LASTEXITCODE -ne 0) { throw 'Plugin syntax check failed' }
}
npm.cmd pack --dry-run --ignore-scripts
if ($LASTEXITCODE -ne 0) { throw 'Package preview failed' }
$routePack = npm.cmd pack --ignore-scripts --json | ConvertFrom-Json
if ($LASTEXITCODE -ne 0) { throw 'Package build failed' }
$routeTarball = Join-Path $routeWork $routePack[0].filename
# 如需发布摘要，可选执行：Get-FileHash -LiteralPath $routeTarball -Algorithm SHA256
Invoke-ComponentDsh --profile $routeProfile --from-default-profile web --dump-config
Invoke-ComponentDsh plugin --profile $routeProfile add $routeTarball
Invoke-ComponentDsh --profile $routeProfile --dump-config
```

包内容检查和自定义业务包的构建要求与第 7.3 节相同；这里的 `--ignore-scripts` 仅适用于已经具备运行产物的包。Skills 示例无需再写 Profile patch。MCP 如需鉴权，将完整 config 写在 `$routeProfileDir/cordis.patch.yml` 的 `team.demo.mcp` 行；组合示例按 C.4 写两条对应 ID。不要套用 HTTP 单插件的 `team.component.demo`。

按第 7.6 节准备模型和工作区后，使用 `Invoke-ComponentDsh --profile $routeProfile --no-open` 启动。根据上表完成对应调用及至少一个失败场景，记录到附录 B，再按第 7.8 节替换业务内容和交付。HTTP 的 `component_health` 不是所有路线都要出现的工具。

8 八个项目的接入与拆分总览

在下表找到自己的项目，将“首次建议”和“首个验收场景”写入接入卡，再按第 2、5、6 章确定入口和包结构。以下是基于已查源码的接入建议，尚未交付的 DSH 包不能按表中拟议名称直接安装。Insight 与 AgentLineage 的组合不改变项目登记，HMigBot 仅作为参考。

8.1 当前形态、首次接入与验收

项目	当前可见的形态	首次建议	DSH 交付	首个验收场景
Insight	会话导入、增量刷新、观测与分析 Web/API/CLI	整理可独立消费的分析 API，与 Core 对齐身份和证据	会话分析 bundle 中的 Insight 插件或 MCP 配置	导入一份会话，查分析结果并回到原文
AgentLineage	Core/Learning SDK、CLI、HTTP 与 Core MCP；已有 DSH HTTP 插件	复用现有接口；补齐与 Insight 的版本和引用映射	同一 bundle 中的 Lineage 插件；Learning 可选	冻结材料，查询证据，检查报告引用
CPX	Agent 与模型 API 之间的捕获代理、CLI	保留代理进程与捕获格式，补生命周期和受控配置	可独立安装的 CPX bundle；不要先做普通查询工具	DSH 第一轮请求经过代理，产出可导入记录
SIFT	CLI、评测/审计、JSON 报告、指导 skill	先接静态分析或已有 transcript 审计；长评测再做任务入口	小 bundle：工具桥接或 MCP，加可选 skill	同一输入经 CLI 与 DSH 得到一致的报告与失败状态
ECAT	迁移/修复循环、评估模块、角色资源、执行器	先确认要复用评估能力还是整套执行；见第 10 章	评估工具与资源，或后续 ACP Agent bundle	先读已有报告或跑一个受控评估场景
DT Verify	仓库和执行方式待补	若为本地验证器，优先 CLI/任务 API；按实际依赖调整	验证工具及必要资源；设备场景增加 runner	输入产物，返回检查项、证据、失败和 skip 原因
Layr	工作流 CLI、版本化程序目录、Opsbot 解释引擎	先接目录与 lint，再通过现有服务接执行；原生 DSH 执行另行适配	Layr bundle；目录/校验与执行可独立装配	校验并执行一个程序，读取节点结果，确认取消与恢复语义
EngramForge	Contracts、typed Client、Host Runtime、OpenClaude Adapter、Center	先接受审核 Release 的检索；宿主采集另行适配，治理留在 Center	Memory bundle；查询与可选采集插件分开，服务独立部署	检索到有版本的已发布记录；候选和未授权数据不可见

Insight 与 AgentLineage 可合为一个业务 bundle，并保留多个插件。CPX、SIFT 先各自做独立的小 bundle，便于其他 profile 复用；业务 profile 再按需组合。不要用“项目较小”判断是否只需一个函数：CPX 的进程和首请求时序、SIFT 的模型评测预算仍需单独处理。

8.2 各项目的初始插件边界

下表给起点，不是要求各项目一次实现所有插件。先按首个可验收场景收窄范围。

项目	建议的首批插件	什么时候继续拆	运行依赖或资产
Insight	一个 `insight-tools` 提供相关分析工具	宿主采集有独立生命周期，或需要单独的 DSH 客户端视图	分析服务、数据存储、可选模型分析
AgentLineage	现有 Core 工具；Learning 当前是配置可选功能	Learning 要独立启停、授权或复用时，再从现插件分离	Core、Learning、材料和快照
CPX	一个 capture 插件处理启停、状态与产物入口	如果控制能力被其他组件单独使用，或后台进程需要独立交付	模型通信代理；首版也可保留外部 launcher
SIFT	一个工具插件或 MCP 配置；附指导 skill	静态检查与动态 eval 要独立开关、预算和执行环境	CLI、评测 runner、数据集和报告
ECAT	按首个目标做评估工具；资源按需装配	完整执行流程需要独立模型、权限和生命周期时，分出 executor	评估器、执行器、角色与流程资源
DT Verify	暂按一个验证入口规划	设备 runner 与离线检查的依赖和生命周期不同才拆	仓库与验证环境待补
Layr	`layr-catalog` 与可选 `layr-runner`；只做 lint 时保留一个插件	生成/转写、目录发布或原生 DSH 执行有独立使用需求	程序目录、解释器、当前 Opsbot 执行服务
EngramForge	`memory-query` 与可选 `memory-capture`	自动上下文注入需独立启停与预算时再分；Center 不默认纳入	Contracts、Client、Runtime、Adapter、Center 与受审核 Release

Insight 与 AgentLineage 先作为同一个业务 bundle 的多个插件。CPX、SIFT、Layr、Memory 可以各自独立交付，再由团队 profile 组合；是否纳入会话分析 bundle，取决于该 bundle 是否承诺提供并维护相应场景。

项目负责人先确认本行的首版范围、外部进程和验收输入，再补实际包名、版本与安装命令。新增项目也按这三项填表，不必重写轻量接入流程。项目数、Plugin 数和 Bundle 数分别统计。

9 Insight 与 AgentLineage 的联合方案

9.1 完整材料与增量材料都需要

两边都围绕会话材料，但不存在固定的“Insight 只要完整会话、AgentLineage 只要 latest”分工。

- 当前 Insight 有 `import-file` 与 `refresh-session`，后者调用 `deltaRefreshSession`，按已有 `sourcePath` 读取变化。
- AgentLineage Core 有完整材料与不可变快照，也有 `live/state`、`live/append`、`live/upsert`、`live/query`、`live/freeze`。
- 当前 Core 的实时 HTTP 接口采用有界增量轮询，没有 SSE。JSONL 通过字节 offset 追加，DevEco 通过 revision 更新；它们不能互相简单替代。

最新界面需要增量视图；可复查的分析需要明确的材料版本或冻结快照。只提交“最近十条消息”通常不够，还要保留此前的上下文、调用与回执、父子会话和采集缺口。Core 增量接口 [S50]

9.2 建议共同使用的处理链

```text
宿主原生会话 / CPX wire 捕获
          ↓ 分别标明来源与版本
采集与材料登记：完整导入或增量提交
          ↓ 共享 session key、材料版本和证据锚点
     ┌────┴──────────────┐
     ↓                   ↓
Insight 分析投影      AgentLineage 原件与证据索引
     ↓                   ↓
指标、异常与报告 ─→ 冻结快照与原文回查
                         ↓ 按需
                Learning 调查与案例草稿
```

建议统一接入入口与材料身份，内部仍允许多个服务。不要要求一个插件通过另一个插件取数据；两个插件消费公共服务。统一入口也不强制先建网关进程，可以先共用接口约定和客户端。

首批至少约定：来源类型和版本、宿主 session ID、项目/运行 ID、父子关系、材料 ID 与摘要、增量 offset/revision/cursor、快照 ID、记录锚点、分析器版本、输入范围与采集缺口。

Insight 的 `turnIndex + label + quote` 和 Core 的 `source_id + sha256 + ref + pointer` 需要显式映射。保留原始坐标，分析结果绑定具体材料版本；不要只凭展示文案匹配证据。`semantic_verified=false`、证据缺失和不完整状态继续保留。

9.3 什么时候增加 Stream API

先分别确定三个需求：生产者怎样提交新材料，分析消费者怎样追赶变化，UI 怎样收到进度通知。它们可以采用不同传输方式。

第一阶段复用 append/upsert 和游标轮询。只有当轮询无法满足约定的延迟、连接数或负载要求时，再增加 SSE。SSE 可通知“有新 revision 或任务进展”，断线后仍依靠持久游标补读；不要把唯一数据副本放在连接里。需要双向连续交互时，再评估 WebSocket 等方案。

增量约定至少覆盖：

情况	应有行为
同一批重复提交	相同身份与内容幂等，不重复计数
乱序、缺块、旧 revision	明确冲突或等待补齐，不静默拼接
源文件替换、截断或回溯修改	识别来源代际；按约定重建或建立新版本
断线重连	从已确认位置补传/补读；游标过期明确要求重同步
两个分析器进度不同	分别返回已处理版本或位置，不声称结果同批完成
暂无新数据	与“会话已经结束”分开表示
来源结束	明确结束标志、最后确认位置、缺口和冻结结果

这张表是联合接入的验收要求，不表示两边已经实现所有行为。完整导入与增量补齐到同一截止点后，比较基础记录和可确定的指标；涉及 LLM 的分析另行评测，不要求生成文本逐字一致。

9.4 现有能力与本次待做的连接

已有代码基础：Insight 导入和刷新、Core 增量与冻结、各自查询分析、AgentLineage DSH HTTP 工具。

联合方案仍需完成：共享会话身份、材料上传/登记约定、证据引用映射、跨模块状态、组合 bundle、DSH 原生会话采集 adapter，以及安装后的真实任务验收。工具插件不自动承担采集职责。

CPX 是模型通信来源，DSH 原生日志是宿主执行来源。两者可以关联，但覆盖范围不同，不能因为捕获文件具有类似 Claude JSONL 的结构就标为 Claude 原生会话。DSH 原生材料应按匹配版本的持久化接口读取，避免猜测其压缩文件布局。

9.5 联合方案按什么顺序实现

顺序	本轮实现什么	进入下一轮的条件
1 完整材料	固定一份脱敏会话，分别跑现有 Insight 导入与 Core 登记/快照	两边记录来源、材料摘要和版本，原文可读
2 引用映射	把一条 Insight 分析定位到 Core 的具体记录	映射有稳定 ID/坐标，缺失时明确返回未定位
3 DSH 组合	各插件调用已验证入口，制作组合 Bundle	干净 Profile 可安装，查询分析并回查同一份证据
4 增量追赶	为一种来源接 append/upsert 与游标，做重复和断线用例	与完整导入到同一截止点的基础记录一致，缺口可见
5 延迟优化	测量轮询延迟和负载，确有需要再增加通知传输	达到接入卡约定指标，断线后仍能补读

每轮留下真实输入、两边输出和验收记录。首版可以停在第 3 轮，交付“已有会话的分析与证据查询”；完成第 4 轮后才声明支持该来源的持续更新。DSH 原生采集、CPX 捕获和 Learning 分别登记范围，不作为全部功能同时上线的前提。

10 各项目的具体注意事项

先按本章收窄首版范围，产出接入卡和一个可复现调用，再回到第 5-7 章实现。项目尚缺入口或环境时，先完成对应前置工作，不用拟议安装命令代替实现。

10.1 CPX

现有 README 已列出 `cpx dsh web` 和 headless 用法，说明 CPX 可以作为外部启动包装；它在 Agent 与模型 API 之间捕获请求与响应，保存记录供 Insight 导入分析。首次先验证这种方式是否覆盖团队所用的 DSH 模型协议、provider 与认证方式，再决定内嵌生命周期插件。CPX 当前说明 [S51]

要达到“普通 DSH 启动后就包含捕获”，需要确认代理先启动、模型连接指向代理、第一轮请求已被捕获、上游与凭据仍正确、退出会刷盘并释放端口。仅新增 cpx_status 之类的工具不会自动完成这些事。不要在 DSH 已启动后再通过工具启动另一个 DSH，来代替当前会话的捕获接入。

建议提供独立与组合两种安装入口。 以下是目标交付设计，尚未据此新建或发布 DSH 包；包名在接入卡中另行确定。

交付物	面向谁、提供什么	边界
CPX 独立 Bundle	需要留存模型通信记录的开发者；配置捕获、状态、记录位置与导出入口	可先保留外部 launcher；完成生命周期适配后再承诺普通 DSH 启动即捕获；不要求 Insight 常驻
Insight + AgentLineage 分析 Bundle	已有会话材料的团队；分析材料、查询证据与追溯原文	支持已有材料导入；不能因为装了工具就宣称已自动采集
可选的会话观测组合 Bundle	希望一次安装捕获、分析与证据查询的用户	复用 CPX 和分析包，声明经过测试的版本组合；CPX 捕获显式启用，Learning 等按需选择

CPX 首版可以只有一个 capture Plugin。状态、产物入口和一个小视图可属于同一职责；只有控制面或视图需要独立启停、复用或不同依赖时再拆。组合 Bundle 的价值是默认装配、配置说明和组合验收，不是再复制一份代理代码。

组合时明确谁拥有每个插件实例。CPX 独立 Bundle 和组合 Bundle 不能各自插入一套相同捕获实例，造成端口冲突或重复记录。可以让组合 Bundle 统一装配各包导出的 Plugin，或者复用已有装配层并明确安装互斥关系；按第 6.7 节选择，两个安装入口都要各自验证。

市场如何介绍 CPX。 建议一句用途写成：“查看 Agent 实际发给模型的上下文，保存可用于排障和离线分析的请求记录。”明确支持的协议、框架和记录范围。wire 请求记录与宿主执行日志覆盖不同，不宣称捕获了所有内部事件，也不把记录能力宣传成已经完成自动诊断或确定性重放。

建议用一项可回读的演示：开启捕获，运行一次真实任务，确认第一轮记录，打开其中一轮请求，再将该份材料交给 Insight 分析，并通过 AgentLineage 定位证据。前半段展示 CPX 独立价值，后半段展示组合价值；尚未连通的映射或跳转标为待实现。

使用者的问题	CPX 应呈现的结果	需要其他组件完成什么
这一轮实际发给模型什么	对应请求、响应与材料位置	Insight 提供阅读和分析视图

使用者的问题	CPX 应呈现的结果	需要其他组件完成什么
为什么上下文增长或请求变慢	已捕获的请求材料与可用时序字段，说明缺失范围	Insight 做对比与分析，不由代理直接断言原因
能否把这次失败交给同事排查	脱敏记录、格式版本与关联标识	AgentLineage 登记材料并提供稳定证据引用

CPX 的持续价值以首轮覆盖率、额外延迟、记录完整性、脱敏与排障耗时衡量。市场存在感取决于用途、首次使用体验和可见结果，不以插件个数或 UI 复杂度衡量。README 应让用户按首次使用指引确认“正在捕获、覆盖什么、文件在哪里、怎样停止”。

捕获脱敏、请求/响应保真、首轮覆盖、异常退出后的文件完整性都应有证据。是否让代理故障阻断请求或允许绕过，作为部署选择明确配置；启停、上游通信和产物读取分别说明责任。

10.2 SIFT

先保留 CLI 与报告格式。可以把不启动被测 Agent 的静态分析接成短调用；已有 transcript 的审计需要先确认其评分器与模型依赖。动态 eval 会启动被测运行，应有任务 ID、执行器选择、预算、取消、产物目录和退出状态。

若通过 DSH 调用本地 CLI，应使用宿主受管执行能力或明确的外部 worker，限定参数和工作目录；不要拼接任意 shell 命令。CLI 的 JSON、退出码、审计结论和基础设施失败要分别保留。

DSH 能调用 SIFT，不等于 SIFT 的被测执行器已经支持 DSH。评估 DSH skills/agents，还需要适配运行器或导入 DSH 的记录格式。数据与评分报告中保留被测版本、模型、样本、失败、超时、skip 和费用。

负责人先选定实际发行来源，再固定一条现有 CLI 命令、一份输入、JSON 输出和退出码；把它们作为 DSH 桥接的对照。选择工具/MCP 路线按第 5.1/5.2 节实现；只交付使用方法则按第 5.3 节做 Skill。当前已查目录有 `D:\work\sift` 和 `cannbot-sentry/packages/sift`，两份 checkout 不能混作同一版本。

10.3 ECAT

当前源码有评估、循环、报告与 `ClaudeCodeRunner`、`OpencodeRunner`。优先让负责人从以下两种目标中选择：

目标	方案	需要确认
让 DSH 使用 ECAT 的评估与改进建议	抽取评估/报告能力，工具调用；流程与角色作为资源	哪些维度调用模型、编译器或设备；DSH 是否承担修复循环
让 DSH 委派一整项 ECAT 任务	保留独立运行时，提供任务 API 或 ACP	会话、进度、权限、预算、取消、产物和失败传播

前者不要偷偷再启一套完整循环；后者要如实说明模型和工具来自 ECAT 的执行器，不能声称全部复用了父 DSH 会话。ECAT 当前存在 runner 抽象不代表已经支持 ACP。

先拿一个已有输入在 ECAT 中跑出报告，列出本轮要暴露的字段、执行成本与依赖。评估工具按第 5.1/5.2 节接入；整任务委派先实现和验证任务入口，再按第 5.4 节处理 ACP。接入卡只能选择已经能验收的路线。

10.4 DT Verify

先补仓库、验证对象、运行命令、结果格式与环境依赖。如果主要是编译/测试/设备检查，建议返回逐项结果和证据，并保留 `pass/fail/skip/error`；缺设备或工具不能算通过。长任务提供状态与取消。如果主要是规范与指引，则按资源包接入。不要仅根据名字预设实现。负责人提交上述材料后，按第 2.2 节选路线；材料未齐前仅登记为待分类，不编造 DSH 包名和启动命令。

10.5 Layr

Layr 当前包含开发 CLI、版本化程序目录和解释引擎，支持 `dag-yaml@1` 与 `skill-md`。CLI 的 lint、目录查看等能力可独立使用；`layr run` 把请求交给 Opsbot 的 `/api/v1/runs`，解释器通过 `KernelContract` 和 `OpsbotKernel` 使用宿主执行能力。现有 `LayrEnginePlugin` 是 Opsbot 插件，不能直接作为 DSH 插件加载。

建议先明确两种接入目标：

目标	方案	需要补齐的工作
DSH 使用现有 Layr 能力	小 bundle 提供目录/lint 工具；执行工具调用现有 Opsbot 服务	服务部署、程序可访问位置、任务 ID、状态、取消与产物
Layr 的节点由 DSH 执行	保留解释器，适配其所需的内核操作或执行后端	session/节点映射、工具与工作区权限、停止/等待、状态持久化、重启恢复

建议的插件分工是 `layr-catalog`（目录、版本、lint）与可选 `layr-runner`（发起运行、状态、停止与产物）。生成、转写、发布目录条目等会用模型或写入资产的操作按需加入，不默认放进只读目录能力。若首版只交付 lint，一个插件即可。

当前 run 提交的是服务端可读取的绝对程序路径，不能直接推广成跨机器上传 API。当前 `--wait` 的 Ctrl-C 只停止等待，不取消服务端 run；DSH 取消需要明确调用后端停止能力并核实任务终态。ACP 只在需要对外交付一个完整 Agent 时考虑，不能自动替代 DAG 调度接口。首轮先以一份固定程序验证目录/lint 的原生调用，再接工具；需要执行时另验现有 Opsbot run，明确服务端路径可访问后再做 runner。

验收选一个小工作流：固定程序版本，验证依赖顺序、节点失败、取消后没有遗留写入、产物可读；宣称恢复能力前再做中断恢复。DSH 工具只负责调用，Layr 继续负责自己的工作流调度，避免两个宿主同时重试同一节点。
Layr CLI [S18]、run 入口 [S52]、引擎适配 [S53]

10.6 EngramForge

EngramForge 提供 Memory 的检索、运行时管理和治理。当前已经有五个独立 Python distribution：Contracts、Client、Runtime、OpenClaude Adapter、Center。DSH bundle 是新增的宿主接入交付物，不应把这五个发行包机械改成五个 DSH 插件。

第一阶段建议做 Memory 查询入口，复用 `QueryClient.search/browse/read`。第二阶段再适配 DSH 的宿主事件与任务生命周期；按需使用 `AdapterIngressClient` 和 `LifecycleClient`，不让模型持有通用 operation dispatcher。Center 继续负责提炼、候选、验证、审核与 Release，不放进普通 Agent 的默认工具集。

```text
DSH 查询插件 → 公共桥接入口 → QueryClient → Host-local Runtime → 已发布 Memory
DSH 采集适配 → 公共桥接入口 → 对应 facade → Runtime 接收/校验事件
                                                    ↓ 按现有同步与治理流程
                                              Center 审核与 Release
```

这是一种拟议接法，公共桥接入口仍需实现。现有 Client 使用 Unix socket 上的 HTTP，并验证 endpoint 所有者与 peer UID；它不是直接填写 `https://...` 就可用的远程 API。本次核对未找到已交付的 DSH 接入包或面向该 Client 的 MCP Server。可在受支持宿主上增加 Python MCP/受控服务桥接，复用 typed Client；若新增网络入口，需重新定义身份、权限和传输保护，不能把本机 UID 信任直接转成远程信任。Windows 原生接入还需验证传输与身份实现，不能因 Python 可安装就认为 Runtime 可用。先在受支持环境用 typed Client 查询一条已发布记录，完成身份与范围验证；再选择 MCP 或受控服务桥接，最后按第 5.2/5.1 节接入 DSH。

建议拆成 `memory-query` 与可选 `memory-capture`。后者适配可信宿主事件和任务身份，不能把任意模型文本伪装为执行事实。两者可先同包发布、独立启用；是否拆为不同包取决于是否独立部署或服务不同使用者。拆插件本身不构成权限隔离，最终仍由 Runtime 校验身份和操作范围。

Raw Inbox 不直接进入检索，候选不等于发布，评测材料不进入提炼输入。正式记录的 `record.json.content` 是内容真相源，Markdown 和检索索引是投影。沿用这些约束，不把 memory 简化成一个任意 `remember(text)` 写入工具。

EngramForge 与 AgentLineage Learning 都涉及经验，但当前没有证据证明可直接互换。组合时先确定谁持有草稿、谁发布正式 Memory、引用怎样关联、撤回如何传播，避免同一结论被两边独立提升为正式知识。首版保留各自职责，通过记录 ID、版本和证据链接关联。

验收至少包括：已审核版本可查、Raw/候选不可查、只读身份不能提交采集事件、错误身份被拒绝、检索范围符合授权、撤回后按协议停止返回、服务不可用时返回可识别状态。模型提炼与业务收益另行验证。Client [S54]、Runtime [S55]、仓库与治理边界 [S56]

11 团队交付与验收

以组件化交付验收表 [S02]作为实施和交付记录，完整内容也见附录 B。AI 读取手册选路线，执行并更新表格；使用者通过结论、阻塞项和证据判断是否可接手。不要只提交“已完成组件化”的总结。

11.1 怎样判定组件化完成

先确定场景、交付版本和本次宿主。默认只验五项：共用实现、调用约定、薄适配、安装使用、实际验证。资源类复用正文/脚本/引用；不要求为它们造 API。已支持的协议可直接配置，不必再写 Driver。

完成 = 范围明确 + L01-L05 全部通过 + 实际触发的 E 项通过 + 结果有证据。

E 项由实际行为触发：服务运行、慢调用/长任务、写入/持久化、增量数据、组合/自有资源、Agent 委派/工作流。只记录选中的扩展；没有扩展时说明一次即可。未知条件写待确认，无法执行写未验证，不因删掉记录就视为通过。

只交付公共能力时验自身入口；声明已完成 DSH 或其他宿主接入时，对每个声明支持的宿主实际跑成功和常见失败。不要把“公共能力可复用”扩大成“所有 Agent CLI 都已支持”。独立复核、干净环境复跑、包摘要、复杂回滚和生产运维由团队按场景增加，不是默认完成条件。

11.2 交给 AI 什么，最终收回什么

给 AI 仓库、接入卡、验收表和所选路线的手册章节。先查现有 SDK/CLI/MCP/资源，再做必要适配；仅在有真实服务需求时新增 HTTP。不要为了填写表格额外建设队列、流式 API 或治理平台。完整任务指令见附录 B.5。

默认交回：版本明确的包或源码交付位置、简短 README、填写后的验收表和真实输入输出。README 写安装/加载、必要依赖与配置、调用例子和常见失败；服务启动、取消恢复、升级回退等只按已选扩展补充。证据可直接放表里，不要求每项另建证据档案。

AI 或人可执行验收，不强制另一名复核者。团队确需独立复核时再追加；没有执行、日志不可读或结果与版本不符的项不能写通过。模板只定义记录和判定规则，不包含自动执行验收的程序。发布、市场收录与业务效果另记状态，不因组件化完成就声称这些工作已完成。

11.3 失败时从哪里查

出现问题时先确认实际 DSH 版本、DSH_HOME 和 Profile，再按下表排查。保留失败命令、插件实例 ID 和日志路径，修复后重跑失败步骤，不从发布或安装环节盲目重来。

现象	优先检查
包安装了但没有能力	bundle 是否被选择；patch 是否插入插件；依赖是否仅安装未加载
插件可见但工具缺失	激活错误、`inject` 服务、目标 Agent 作用域、禁用与上层覆盖
MCP 连接失败但 Web 正常	MCP 激活状态、地址/传输/鉴权、服务日志与真实工具列表
Skill 能列出但不能执行	正文读取、资源根、工具映射、脚本环境与实际权限
Agent 角色文件存在但不能派发	preset/persona 或定制路由是否装配，执行后端及调用入口是否存在
ACP 任务被拒绝	协商能力、provider 限制、权限策略、外部运行时与凭据
CPX 没捕获第一轮	启动顺序、模型 endpoint 和代理端口是否真正生效
增量分析重复或缺记录	session/source 身份、确认 offset/revision、幂等和材料代际
升级后行为没变化	实际安装副本、包版本/摘要、是否重启、是否仍用了旧配置

12 本版依据和待确认项

原生安装命令按 DSH `0.2.1-alpha.2`、本地快照 `d74326738864` 的 CLI、插件管理器和 Web 插件页核对；2026-10-15 同时查阅上游文档。原生安装支持 npm/Git/tarball/本地包，不依赖社区市场。创建 Profile 的版本差异见第 7.1 节。市场资料核对于 2026-10-14；各站统计和收录政策可能变化。

四个示例包在 v0.7 已完成实际打包、语法及文件检查，并本地核对示例 HTTP 服务的成功、失败与鉴权场景。本次 v0.9 改为五项轻量要求及按需扩展，统一入口选型、接入卡、验收表和 AI 指令；公共能力不强制 HTTP 服务。检查模板同步、默认要求一致性、文档链接、命令语法和 PDF。示例实现未变，不重复执行其业务检查。未执行远程插件安装、npm publish、GitHub Release 或市场提交。当前环境尚未完成真实 DSH 启动、模型调用和 UI 验收。各项目新增安装入口及组合 Bundle 仍是建议；DeepSeek.club 后台审核与自动采集尚未验证。

主要参考：

- 用户指定的 PPT [S57]，以及本次核对补充的 Skills & Agents、ACP 范围。
- 现有 DSH 101 讲稿 [S58]与 Insight 联合设计 [S59]。后者的旧盘点不能替代当前源码；本次核对 Insight 注册表已包含 `codex-jsonl`。
- DSH CLI 与 profile [S47]、CLI 配置层规则 [S42]、工具开发 [S60]、插件管理与兼容性 [S19]。
- Agent preset [S09]、persona [S61]、ACP 服务 [S62]。
- Insight 来源注册 [S63]、增量刷新 [S64]、CPX [S51]、SIFT [S65]、ECAT 执行器 [S66]、HMigBot DSH [S67]。
- Layr 引擎装载 [S68]、KernelContract [S69]、EngramForge 独立发行包 [S17]、Client facade [S70]、Unix socket 传输 [S71]。

查阅快照：AgentLineage `9ac5d054f26f`；DSH `d74326738864`；HMigBot `b4af8584f680`；cannbot-sentry `98d717a2b29e`；独立 SIFT `52e447ae929b`；ECAT `c1a1fb9a43ec`；Layr `275515214698`；EngramForge `b9e591eb8982`。这些是本地 checkout 定位，不表示远端最新状态。

待确认：各项目负责人、DT Verify 仓库、SIFT 发布来源、团队实际部署的 DSH 版本、联合 bundle 的首批功能和实时延迟目标、ECAT 的接入目标、Layr 采用服务调用还是 DSH 原生执行、EngramForge 首版是否只开放检索及目标操作系统。确认后只需补项目接入卡及对应章节，不必重写通用流程。

附录 A 组件接入卡

先填基本信息和最小样例即可开始。其他字段只填实际涉及的部分；保留现有入口，不要求为组件化新建 HTTP 服务。未知项写待补，建议项写建议。

A.1 基本信息

字段	填写内容
组件、负责人、版本	仓库与包/源码版本或提交
首个场景	给什么输入，完成什么，得到什么结果
已有入口与选择	SDK / CLI / MCP / HTTP / Skills & Agents / ACP；本次复用哪个，为什么
本次宿主与范围	声明支持哪些 Agent CLI；只交付公共能力时写明；暂缓项是什么
业务与适配位置	共用实现在哪里；宿主差异在哪里；直接协议接入可写仅配置
必要环境与配置	实际需要的运行时、文件、工具、模型或服务；不填凭据原文
扩展选择	按验收表选择实际触发的 E 项；没有时写无及简短依据

A.2 最小样例

输入或调用命令：待补。预期输出：待补。

一个常见失败及预期表现：待补。

已有类型定义、CLI 帮助、工具 schema 或资源说明位置：待补。可直接引用，不必另写完整接口规范。

A.3 宿主接入与交付

只填本次宿主。DSH 接入时写下表；其他宿主写对应安装和调用方式。

项目	填写内容
交付物与版本	包/源码地址、实际验证版本；内部交付文件也可以，不要求上架
DSH 装配	Plugin/Bundle 包名、实例 ID、目标 Profile、所需宿主服务；按实际使用填写
安装与配置	实际安装/加载命令、依赖和配置位置；DSH 用户配置写 Profile，不改包内默认 patch
调用与结果	使用者怎样调用，怎样确认成功，失败时先查什么
拆分与复用	没有独立职责就保持一个插件；按需说明多入口、多包或组合方式

A.4 按需补充

从下列内容选与 E 项有关的部分，未涉及的部分不要求填写或实现。

触发情况	补充内容
服务/常驻进程	谁启动、配置地址与访问边界；谁停止，怎么判断不可用
慢调用/长任务	超时、中断、结果取得；需要后台执行才补状态与恢复
写入/持久化	写哪里、重复执行的后果、失败恢复；已有数据变更时补迁移/回退

触发情况	补充内容
增量数据	来源、offset/revision/cursor、重复处理、断线和结束条件
组合/委派	可选开关、依赖和实例归属；自有资源清理；任务结果与失败回传
对外发布	分发渠道与安装地址、发布身份与许可；摘要按发布流程要求；市场收录单独登记

A.5 交付记录

交回版本明确的交付物、简短 README、成功/失败结果和轻量验收表。L01-L05 及实际触发的扩展通过且有证据，才判定本次范围完成。独立复核、生产化检查和业务效果评测按团队需要另加。

需要拆分或发布时查发布和拆分指南，不要求先完成公开发布才能做组件化交付。

附录 B 验收记录

规则 v2 · 配套手册 v0.9。默认目标是复用业务能力、减少各 Agent CLI 的重复实现。公共入口可以是 SDK、CLI、MCP 或资源包；不要求新增 HTTP 服务，也不要求实现所有入口。

B.1 本次范围

字段	填写内容
组件与范围	组件名、负责人；首个场景、交付能力与明确排除项
入口与宿主	复用哪个现有入口；本次声明支持哪些宿主；只交付公共能力时写明不包含宿主接入
版本与环境	包/源码版本或提交、交付位置、必要依赖与配置、实际验证环境；凭据只写变量名
按需扩展	写实际触发的 E 项与原因；无扩展时简述为何不涉及，不逐项填写不适用

B.2 五项轻量要求 L01-L05

每项填“通过 / 失败 / 未验证”和实际证据。源码检查可支持结构判断；安装、调用和失败处理必须有实际执行结果。L 项保留全部五行；只交付公共能力时按其自身入口验证，不强行增加 DSH。

ID / 检查	通过条件	状态	证据 / 实际结果
L01 共用实现	业务逻辑只维护一份，不在各宿主插件中复制；资源类复用同一套正文、脚本和引用	未验证	待补
L02 调用约定	写清输入、输出、常见错误，附一个可运行例子；可复用函数类型、CLI 帮助或工具 schema，无须另造协议	未验证	待补
L03 薄适配	Driver 只做注册、参数转换、调用和结果展示；无宿主适配时检查公共层不依赖宿主；已有协议可直接配置，不强造 Driver	未验证	待补
L04 安装使用	有明确版本、依赖、必要配置和安装/加载及调用命令，按说明实际可用；交付 DSH 包时包含所需入口、patch 与资源	未验证	待补
L05 实际验证	本次声明支持的每个宿主分别跑通一个成功场景和一个常见失败场景；只交付公共能力则验证该入口；保留真实输入输出	未验证	待补

B.3 触发时才增加的检查

以下是选择表，不是默认待办。按实际行为选择，不能仅因使用 MCP、Skills 或 SDK 就要求全部扩展。普通 stdio MCP 不需要对外部署 HTTP 服务；角色资源按 L 项验证即可。

扩展	什么时候需要	只增加什么
E01 服务运行	本次确实启动或依赖常驻/远程服务	启动者、地址/访问边界、不可用时的表现；自己管理进程才验停止清理。远程按实际需要验鉴权，不要求多租户或 HA
E02 慢调用或长任务	操作可能长时间等待或后台持续执行	超时、中断和结果取得方式；只有确实需要后台任务才设计任务 ID、状态和恢复，不强制上队列

扩展	什么时候需要	只增加什么
E03 写入与持久化	会修改文件/数据库、发布数据或保留状态	副作用、重复执行、失败恢复；涉及已有数据格式变化时才补迁移/回退。自动重试不是必需功能
E04 增量数据	持续追加、刷新或断线续读	来源/位置标识、重复处理、断线与完整性；按实际格式核对结果，不强制 SSE/WebSocket
E05 组合与资源	多插件组合、可选功能，或自己持有子进程/连接	实际组合不重复注册；可选项按声明启停；自有资源能清理，不要求不存在的组合方式
E06 Agent 委派/工作流	执行独立 Agent、多步流程或外部运行器	输入输出与执行归属、失败/取消和产物；预算、并发与权限只验本次声明的能力

只为选中的扩展增加结果行。实现中发现新行为触发扩展时补选；不确定是否触发就写待确认，不通过删行绕过真实问题。

选中 ID	本次要验证什么	状态	证据 / 实际结果
待填，或写“无”	只写实际适用内容	未验证	待补

B.4 完成判定与证据

完成 = 范围明确 + L01-L05 全部通过 + 实际触发的扩展通过 + 结果有证据。 没有执行、缺环境或证据不可读，记未验证；失败或未验证不能写完成。范围变更留一句说明，并重验受影响项。

证据可直接写在表里：命令/操作、预期、实际结果或日志/产物位置，并注明版本与环境。一次执行可支持多项。无需为每一行另建材料；模型声称成功不能代替调用结果。

结论字段	填写内容
本次结论	未完成；满足规则后写“本次范围内组件化完成”，列明已验证的宿主/入口和版本
未完成项	ID、原因、下一步；无阻塞时填“无”
可选加强	独立人/AI 复核、干净环境复跑、包摘要、升级/回滚专项等，由团队按场景选择，不作为统一门槛
单独记录	若包含发布、市场收录或业务效果承诺，分别写状态与证据；不等同于组件化完成

B.5 给 AI 的实施与验收指令

```text
读取仓库和接入卡，先确定场景、已有入口、本次宿主与交付版本。
优先复用现有 SDK/CLI/MCP/资源；只有实际需要才新增 HTTP 服务。
业务逻辑维护一份；已有协议能接入就用配置，否则写必要的薄适配。
按 L01-L05 实施和验收；实际触发 E 项才补检查，不扩大首版范围。
保留必要依赖和调用说明，不为了表格新增队列、流式 API 或治理系统。
实际执行安装/加载、成功调用与常见失败；逐项写结果及证据。
未执行记未验证并列阻塞；不把推测、代码存在或工具可见写成通过。
按本附录 B.4 规则输出完成/未完成；附交付物和表格。独立复核按团队需要。
```

附录 C 最小接入示例

先按第 5 章选择路线，再复制对应的完整目录，不要把所有示例装进同一 Profile。以下列出 HTTP、MCP、Skills、同包多插件四种示例交付物，以及独立 HTTP 测试服务。目录保留在 examples 下；重复的代码在文中明确引用，真实示例目录含完整文件。示例的静态、打包和本地服务检查不代替真实 DSH 宿主验收。

C.1 HTTP 单插件 Bundle

本包演示一个 npm 包同时提供 plugin 和 bundle。它使用 DSH `0.2.1-alpha.2` 源码中的工具契约，经过 JS 语法与打包内容静态检查，尚未在真实 DSH 宿主安装运行。目标版本不同，应先调整并验证兼容性。

配置 `baseUrl`；可选 `tokenEnv` 是已有环境变量的名称。示例要求独立服务提供 `GET <baseUrl>/health`，返回一个很小的 JSON，例如 `{"status":"ok"}`。该地址和接口是示例约定，不是 Insight 或 AgentLineage 的现有接口。调用传递取消信号、设置五秒超时并拒绝重定向。

本包不包含 HTTP 服务，不会自行启动服务。C.5 提供单独的本地示例服务，由另一个终端启动。没有配置 token 时可连接本机不需要认证的测试服务；真实服务按其鉴权要求配置。示例只校验 health 结果，不是可直接推广到任意响应体的通用 HTTP 客户端。

在包含该包 package.json 的 `examples/http-bundle` 目录执行：

```powershell
node --check index.js
npm.cmd pack --dry-run --ignore-scripts
npm.cmd pack --ignore-scripts
```

安装和配置按主指南第 7 章执行。工具名为 `component_health`，成功返回服务实际报告的 `status`。`degraded` 等状态需要按原值解释，不能一律视为业务通过。

修改为自己的组件时，替换包名、实例 ID、工具 schema 与调用，明确失败和取消语义，加入接口需要的响应大小限制和结果校验。按团队交付要求补图标；此示例包使用宿主默认图标。不要把本包的 health 验证当成业务集成完成。

http-bundle/package.json

```json
{
  "name": "@team/component-demo",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "description": "Teaching example: expose an HTTP health check as a DSH tool",
  "engines": { "node": "^22.19.0 || >=24.0.0" },
  "exports": {
    ".": "./index.js",
    "./locale/*.json": "./locale/*.json"
  },
  "files": ["index.js", "cordis.patch.yml", "README.md", "locale"],
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } },
  "peerDependencies": { "@deepseek-ai/dsh-tools": "0.2.1-alpha.2" }
}
```

http-bundle/cordis.patch.yml

yaml

```yaml
- insert:
    - id: team.component.demo
      name: '@team/component-demo'
      config:
        baseUrl: http://127.0.0.1:8120
```

http-bundle/index.js

javascript

```javascript
import { defineTool } from '@deepseek-ai/dsh-tools';

export const name = 'component-demo';
export const inject = ['tools'];

// Teaching contract: GET <baseUrl>/health returns a small JSON object
// with a string status. Configuration belongs to the profile, not tool input.
export function apply(ctx, config = {}) {
  if (typeof config.baseUrl !== 'string') {
    throw new Error('component-demo: baseUrl is required');
  }
  const base = new URL(config.baseUrl.endsWith('/')
    ? config.baseUrl : config.baseUrl + '/');
  const local = ['127.0.0.1', 'localhost', '[::1]'].includes(base.hostname);
  if (base.username || base.password || base.search || base.hash
      || !(base.protocol === 'https:' || (base.protocol === 'http:' && local))) {
    throw new Error('component-demo: use HTTPS or loopback HTTP without credentials/query/hash');
  }
  const headers = { Accept: 'application/json' };
  if (config.tokenEnv !== undefined) {
    if (typeof config.tokenEnv !== 'string' || !config.tokenEnv
        || !process.env[config.tokenEnv]) {
      throw new Error('component-demo: configured token environment variable is missing');
    }
    headers.Authorization = `Bearer ${process.env[config.tokenEnv]}`;
  }
  const endpoint = new URL('health', base);
  const tool = defineTool({
    name: 'component_health',
    description: 'Read the configured example service health status. Does not start the service.',
    parameters: {},
    output: {
      schema: { type: 'json' },
      render: (_args, value) => [{ type: 'text', text: JSON.stringify(value) }],
    },
    async execute(_args, exec) {
      const response = await fetch(endpoint, {
        headers,
        signal: AbortSignal.any([exec.signal, AbortSignal.timeout(5000)]),
        redirect: 'error',
      });
      if (!response.ok) throw new Error(`component-demo: HTTP ${response.status}`);
      const body = await response.json();
      if (!body || typeof body.status !== 'string' || body.status.length > 256) {
        throw new Error('component-demo: invalid health response');
      }
      return { status: body.status };
```

```
javascript · 续

    },
  });
  ctx.effect(() => ctx.tools.register(tool));
}
```

http-bundle/locale/en.json

```
json

{ "meta": { "title": "HTTP integration example", "description": "Expose the configured service health status
   as a DSH tool." } }
```

http-bundle/locale/zh.json

```
json

{ "meta": { "title": "HTTP 接入示例", "description": "把已配置服务的健康状态注册为 DSH 工具。" } }
```

C.2 MCP 配置型 Bundle

本包只有装配配置，没有自定义 DSH 运行代码，也不包含 MCP 服务。它参考 DSH `0.2.1-alpha.2` 随附的 MCP bundle 模板，经过 JSON/YAML 与包内容静态检查，尚未安装到真实宿主。

前提：目标 DSH 能解析匹配版本的 `@deepseek-ai/dsh-mcp-client`，组件已经有一个可连接的 MCP 服务。这里的 `http://127.0.0.1:8120/mcp` 是占位配置，必须替换或启动对应测试服务。

在本目录执行 `npm pack --dry-run --ignore-scripts` 核对文件，再执行 `npm pack --ignore-scripts`。按第 7.9 节创建测试 profile、安装实际 tarball、检查配置并启动，不套用 HTTP health 服务。工具来自 MCP 服务的真实发现结果，不由本包凭空生成；名称形如 `mcp__component_demo__ping`，其中 `ping` 只有服务确实提供时才存在。

若服务需要 token，在 profile 的完整 `config` 中增加：

```
yaml

headers:
  Authorization: !!js '`Bearer ${process.env.COMPONENT_MCP_TOKEN}`'
```

从安全配置渠道设置变量，不把值写进包或聊天。修改 `config` 时保留 serverName、transport、url 等全部必需字段。若使用 stdio，改用该传输的 command/args/env/cwd 字段，并验证进程启动和退出。

真实验收包括发现并调用工具、服务不可用、凭据错误、取消与重连。初始激活失败不必然阻止整个 Web 启动，应单独检查插件。正式交付补组件名称、文档和图标；此示例包使用默认图标。

mcp-bundle/package.json

```json
{
  "name": "@team/component-mcp-demo",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "description": "Teaching example: connect an existing MCP service to DSH",
  "exports": { "./locale/*.json": "./locale/*.json" },
  "files": ["cordis.patch.yml", "README.md", "locale"],
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } },
  "peerDependencies": { "@deepseek-ai/dsh-mcp-client": "0.2.1-alpha.2" }
}
```

mcp-bundle/cordis.patch.yml

```yaml
- insert:
    - id: team.demo.mcp
      name: '@deepseek-ai/dsh-mcp-client'
      config:
        serverName: component_demo
        transport: streamable-http
        url: http://127.0.0.1:8120/mcp
        failOnStartupError: true
```

mcp-bundle/locale/en.json

```json
{ "meta": { "title": "MCP integration example", "description": "Connect an existing tool service through the
   DSH MCP client." } }
```

mcp-bundle/locale/zh.json

```json
{ "meta": { "title": "MCP 接入示例", "description": "通过 DSH 内置 MCP client 连接已有工具服务。" } }
```

C.3 Skills 资源 Bundle

该示例无 HTTP 服务依赖，按第 7.2、7.9 节的 Skills 路线直接操作，源包目录为 `examples/skills-bundle`。默认使用标准 preset，要求加载 component-check 并读取检查码。随包资源的 frontmatter 和相对目录都要保留。

skills-bundle/package.json

```json
{
  "name": "@team/component-skills-demo",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "description": "Teaching example: load packaged skills with DSH's filesystem provider",
  "exports": { ".": "./index.js", "./locale/*.json": "./locale/*.json" },
  "files": ["index.js", "cordis.patch.yml", "assets", "locale", "README.md"],
  "dsh": { "bundle": { "patch": "./cordis.patch.yml" } },
  "peerDependencies": { "@deepseek-ai/dsh-skill-filesystem": "0.2.1-alpha.2" }
}
```

skills-bundle/cordis.patch.yml

```yaml
- insert:
    - id: team.component.skills
      name: '@team/component-skills-demo'
      config: {}
```

skills-bundle/index.js

```javascript
import { fileURLToPath } from 'node:url';
import { apply as mountSkills } from '@deepseek-ai/dsh-skill-filesystem';

export const name = 'component-skills-demo';
export const inject = ['skills'];

export function apply(ctx) {
  mountSkills(ctx, {
    providerName: 'team-component-demo-skills',
    includeDefaultRoots: false,
    bundledSkillDir: fileURLToPath(new URL('./assets/skills/', import.meta.url)),
    watch: false,
  });
}
```

skills-bundle/assets/skills/component-check/SKILL.md

```markdown
---
name: component-check
description: Read a packaged checklist and explain what counts as a successful component check.
---

# Component check

Read `references/checklist.md` relative to this skill's resource directory.
Explain the checklist in Chinese. Include the reference's check code so the user can verify it was read.
This skill does not provide a health tool or start a service.
Only if the user separately asks for a live service check and `component_health` is available, call that too
  l and report its actual result.
Never infer a live check from reading this document.
```

skills-bundle/assets/skills/component-check/references/checklist.md

```markdown
# Checklist

Check code: DSH-SKILL-REF-01

- The service is reachable.
- The actual tool result has a recognized status.
- A failed or degraded status is reported as such.
- Reading instructions alone does not verify a live service.
```

skills-bundle/locale/en.json

```json
{ "meta": { "title": "Component skill example", "description": "Load a packaged skill and its reference file
  ." } }
```

skills-bundle/locale/zh.json

```json
{ "meta": { "title": "组件 Skill 示例", "description": "加载随包交付的 Skill 和相对引用资料。" } }
```

C.4 HTTP 与 Skill 同包多插件

源码位于 `examples/http-skills-bundle/`。其中 http.js 采用 C.1 的完整 index.js，skills.js 采用 C.3 的完整 index.js，assets/skills 与 C.3 相同。以下清单和 patch 负责把它们装进一个包；真实目录中已包含所有文件，无须读者拼接缺失模块。locale/README 亦随包交付，示例子入口使用默认显示回退。

http-skills-bundle/package.json

```json
{
  "name": "@team/component-http-skills-demo",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "description": "HTTP tools and optional packaged skills in one DSH bundle",
  "exports": {
    "./health": "./http.js",
    "./skills": "./skills.js",
    "./locale/*.json": "./locale/*.json"
  },
  "files": [
    "http.js",
    "skills.js",
    "assets",
    "locale",
    "cordis.patch.yml",
    "README.md"
  ],
  "dsh": {
    "bundle": {
      "patch": "./cordis.patch.yml"
    }
  },
  "peerDependencies": {
    "@deepseek-ai/dsh-tools": "0.2.1-alpha.2",
    "@deepseek-ai/dsh-skill-filesystem": "0.2.1-alpha.2"
  }
}
```

http-skills-bundle/cordis.patch.yml

```yaml
- insert:
    - id: team.component.combo.health
      name: '@team/component-http-skills-demo/health'
      config:
        baseUrl: http://127.0.0.1:8120
    - id: team.component.combo.skills
      name: '@team/component-http-skills-demo/skills'
      disabled: true
      config: {}
```

用新 profile 安装该包后，在 profile patch 中启用可选资源：

```yaml
- id: team.component.combo.skills
  disabled: false
  config: {}
```

安装步骤见第 7.9 节，使用独立 Profile。首次安装已由 bundle 提供 health 的默认 baseUrl。需要改地址时，另用 `team.component.combo.health` 覆盖完整 config。分别测试“只有 health”和“health + skills”，不把一个实例的成功当成两个插件都通过。

C.5 HTTP 示例服务

保存为 `examples/fixtures/health-server.mjs`。它只用于第 7 章的本地接入验证，正常、异常、慢响应通过进程启动环境选择；它不代表业务系统，也不实现 MCP。

fixtures/health-server.mjs

```javascript
import { createServer } from 'node:http';

// Teaching fixture, not a production component service or an MCP endpoint.
const port = Number(process.env.DEMO_PORT ?? 8120);
const mode = process.env.DEMO_MODE ?? 'ok';
const modes = ['ok', 'degraded', 'invalid', 'http-error', 'slow'];
if (!Number.isInteger(port) || port < 1 || port > 65535 || !modes.includes(mode)) {
  throw new Error('Use DEMO_PORT=1..65535 and DEMO_MODE=ok|degraded|invalid|http-error|slow');
}
const server = createServer((req, res) => {
  const send = (code, body) => {
    if (res.destroyed) return;
    res.writeHead(code, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(body));
  };
  if (req.method !== 'GET' || req.url !== '/health') return send(404, { error: 'not_found' });
  if (process.env.COMPONENT_TOKEN
      && req.headers.authorization !== `Bearer ${process.env.COMPONENT_TOKEN}`) {
    return send(401, { error: 'unauthorized' });
  }
  console.log(`GET /health mode=${mode}`);
  if (mode === 'http-error') return send(503, { error: 'demo_unavailable' });
  if (mode === 'invalid') return send(200, { ready: true });
  if (mode === 'slow') {
    const timer = setTimeout(() => send(200, { status: 'ok' }), 6500);
    res.on('close', () => clearTimeout(timer));
    return;
  }
  send(200, { status: mode });
});
server.on('error', error => { console.error(error.message); process.exitCode = 1; });
server.listen(port, '127.0.0.1', () => {
  console.log(`Ready: http://127.0.0.1:${port}/health (${mode})`);
});
for (const signal of ['SIGINT', 'SIGTERM']) {
  process.on(signal, () => { server.close(); server.closeAllConnections(); });
}
```

附录 D 资料与源码定位

本地源码按本文 checkout 定位；原生安装说明核对于 2026-10-15，在线市场资料核对于 2026-10-14。在线事实与站方自述分别按正文标注，链接不代表推荐或已实测。PDF 正文的来源编号可跳到本表。

[S01] 接入卡模板

AgentLineage/docs/componentization/component-card.template.zh-CN.md

[S02] 轻量验收表

AgentLineage/docs/componentization/acceptance-record.template.zh-CN.md

[S03] plugin.ts

AgentLineage/integrations/dsh/src/plugin.ts

[S04] client.ts

AgentLineage/integrations/dsh/src/client.ts

[S05] bundle patch

AgentLineage/integrations/dsh/cordis.patch.yml

[S06] DSH MCP client

deepseek-harness/packages/mcp/mcp-client/README.zh.md

[S07] MCP 传输规范

https://modelcontextprotocol.io/specification/2025-06-18/basic/transports

[S08] DSH skill-filesystem

deepseek-harness/packages/skill/skill-filesystem/README.zh.md

[S09] Agent preset 使用契约

deepseek-harness/packages/preset/agent-preset/README.zh.md

[S10] Persona 作用域实现

deepseek-harness/packages/preset/persona/src/index.ts

[S11] ACP 协议概览

https://agentclientprotocol.com/protocol/v1/overview

[S12] DSH ACP 子 Agent 后端

deepseek-harness/packages/subagent/subagent-acp/README.zh.md

[S13] 委派工具配置

deepseek-harness/packages/subagent/tool-subagent/README.zh.md

[S14] DSH 安装实现

deepseek-harness/packages/boot/plugin-manager/src/operations.ts

[S15] Awesome 收录口径

https://github.com/awesome-dsh-plugin/awesome-dsh-plugin

[S16] dsh-market 包清单

https://github.com/dsh-market/dsh-market/blob/main/package.json

[S17] EngramForge 发行单元

EngramForge/docs/deployment/memory-distributions.md

[S18] Layr 项目说明

layr/README.md

[S19] 官方插件管理机制

deepseek-harness/packages/boot/plugin-manager/README.zh.md

[S20] dsh.pub 口径

https://dsh.pub/en/guide/

[S21] dsh.so 说明

https://www.dsh.so/

[S22] dsh-market / dshmarket.com

https://github.com/dsh-market/dsh-market

[S23] dsh.pub

https://github.com/dsh-pub/dsh-pub

[S24] dsh-ai.org

https://dsh-ai.org/sources

[S25] DeepSeek.club Plugin Hub

https://deepseek.club/plugins

[S26] DSH Directory

https://dsh.directory/

[S27] DSH Get

https://github.com/bobby-sheng/dshget-data

[S28] DSHX

https://dshx.io/en/plugins

[S29] dsh-plugin.net

https://dsh-plugin.net/

[S30] dshplugins.co

https://dshplugins.co/zh/

[S31] dsh-plugin.shop

https://www.dsh-plugin.shop/

[S32] LivXue/dsh-plugin-shop

https://github.com/LivXue/dsh-plugin-shop

[S33] bradeGithub/DSH-Plugins-Marketplace

https://github.com/bradeGithub/DSH-Plugins-Marketplace

[S34] dsh-find-plugin

https://github.com/awesome-dsh-plugin/dsh-find-plugin

[S35] 社区主页

https://deepseek.club/

[S36] 提交入口

https://deepseek.club/plugins/submit

[S37] 公开提交表单实现

https://deepseek.club/assets/PluginSubmitPage-DPb1cM2T.js

[S38] 列表实现

https://deepseek.club/assets/PluginsPage-DuLKZsoR.js

[S39] Skill 条目示例

https://deepseek.club/plugins/dsh-esports-ar-tour-skill

[S40] npm publish 规则

https://docs.npmjs.com/cli/v11/commands/npm-publish/

[S41] 贡献指南

https://github.com/awesome-dsh-plugin/awesome-dsh-plugin/blob/main/contributing.md

[S42] CLI 配置层规则

deepseek-harness/apps/cli/reference/README.zh.md

[S43] 上游 Profile 创建与重置说明

https://github.com/deepseek-ai/deepseek-harness/blob/master/apps/cli/reference/README.zh.md

[S44] 内置插件页

deepseek-harness/packages/client/ui-plugin-manager/README.zh.md

[S45] DSH 运行入口

deepseek-harness/README.zh.md

[S46] CLI 选项位置

deepseek-harness/docs/upgrade-guide/v0.2.1-alpha.1/plugin-option-placement/guide.zh.md

[S47] Profile 与合成规则

deepseek-harness/apps/cli/README.zh.md

[S48] 模型配置与端点

deepseek-harness/packages/llm/llm-deepseek/README.zh.md

[S49] Web 启动与模型入口

deepseek-harness/packages/bundle/web-app/README.zh.md

[S50] Core 增量接口

AgentLineage/docs/interfaces/live-streams-v1.md

[S51] CPX 当前说明

cannbot-sentry/packages/cpx/README.md

[S52] run 入口

layr/src/layr/cli/run.py

[S53] 引擎适配

layr/src/layr/engine/adapter.py

[S54] Client

EngramForge/distributions/memory-client/README.md

[S55] Runtime

EngramForge/distributions/memory-runtime/README.md

[S56] 仓库与治理边界

EngramForge/docs/repository-architecture.md

[S57] 用户指定的 PPT

ppt-projects/dsh-components_ppt169_20261011/exports/DSH-组件关系与接入方案-HTTP接口与DSH映射.pptx

[S58] 现有 DSH 101 讲稿

AgentLineage/docs/talks/2026-10-13-dsh-101-hmigbot-agentlineage.zh-CN.md

[S59] Insight 联合设计

AgentLineage/docs/design/2026-10-12-insight-agentlineage-integration.zh-CN.md

[S60] 工具开发

deepseek-harness/docs/cookbook/adding-a-tool.md

[S61] persona

deepseek-harness/packages/preset/persona/README.zh.md

[S62] ACP 服务

deepseek-harness/packages/acp/acp/README.zh.md

[S63] Insight 来源注册

cannbot-sentry/packages/insight/src/lib/ingest/adapters/registry.ts

[S64] 增量刷新

cannbot-sentry/packages/insight/src/app/api/ingest/refresh-session/route.ts

[S65] SIFT

cannbot-sentry/packages/sift/README.md

[S66] ECAT 执行器

ECAT/src/ecat/runner/__init__.py

[S67] HMigBot DSH

hmigbot-dsh/README.md

[S68] Layr 引擎装载

layr/src/layr/engine/plugin.py

[S69] KernelContract

layr/src/layr/engine/contract.py

[S70] Client facade

EngramForge/distributions/memory-client/src/engramforge_memory_client/facades.py

[S71] Unix socket 传输

EngramForge/distributions/memory-client/src/engramforge_memory_client/_uds.py